

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE
PETRÓLEO
CENTRO DE CIÊNCIA E TECNOLOGIA

PROJETO DE ENGENHARIA
DESENVOLVIMENTO DO SOFTWARE
FERRAMENTA PARA GERAÇÃO DA CURVA DE FLUXO
FRACIONÁRIO
DISCIPLINA LEP-0144: Projeto de Software Aplicado à
Engenharia
Setor de Modelagem Matemática Computacional

Versão 1:
Fabiane da Silva Barros
Prof. André Duarte Bueno

MACAÉ - RJ
Novembro - 2025

Sumário

1	Concepção	7
1.1	Metodologia	7
1.2	Nome do Sistema/Produto	8
1.3	Especificação	8
1.4	Requisitos	9
1.5	Casos de Uso	10
2	Elaboração	12
2.1	Análise de domínio	12
2.2	Formulação teórica	13
2.3	Identificação de pacotes – assuntos	15
2.4	Diagrama de pacotes – assuntos	16
3	AOO – Análise Orientada a Objeto	17
3.1	Diagramas de classes	17
3.2	Diagrama de sequência – eventos e mensagens	19
3.3	Diagrama de comunicação – colaboração	21
3.4	Diagrama de estado	22
3.5	Diagrama de atividades	23
4	Projeto	25
4.1	Projeto do sistema	25
4.2	Projeto orientado a objeto – POO	26
4.3	Diagrama de componentes	27
4.4	Diagrama de implantação	28
5	Ciclos de Planejamento/Detalhamento	32
5.1	Ciclo 1 (Versão 0.1) - MVP: Núcleo de Cálculo e Modelo Tabular	32
5.2	Ciclo 2 (Versão 0.2) - Extensão: Modelo Analítico Corey	33
5.3	Ciclo 3 (Versão 0.3) - Validação Científica: Tarek Ahmed	33
5.4	Ciclo 4 (Versão 0.4) - Apresentação: Integração Gnuplot	34
6	Ciclos Construção - Implementação	35
6.1	Código fonte	35

7	Teste	51
7.1	Teste 1: Validando o Ciclo 1 - Modelo Tabelado	51
7.2	Teste 2: Validando o Ciclo 2 - Modelo Corey	53
7.3	Teste 3: Validação com Literatura (Tarek Ahmed)	55
7.4	Teste 4: Validação da Camada de Apresentação (Gnuplot)	58
8	Documentação para o Desenvolvedor	60
8.1	Dependências para compilar o software	60
8.2	Como gerar a documentação usando doxygen	61
A	Formato do Arquivo de Entrada	65
A.1	Exemplo de Arquivo de Entrada (Modelo Tabelado)	65
A.2	Exemplo de Arquivo de Entrada (Modelo Corey)	66
B	Guia de Compilação e Execução	67
B.1	Pré-requisitos	67
B.2	Compilação do Projeto	67
B.3	Execução e Teste	68

Lista de Figuras

1.1	Metodologia utilizada no desenvolvimento do sistema.	7
1.2	Diagrama de caso de uso – Caso de uso geral	11
1.3	Diagrama de caso de uso específico – Executar Cálculo.	11
2.1	Curva de fluxo fracionário (f_w) característica, mostrando a sua forma de "S". Fonte: Adaptado de Ahmed, 2019 [Ahmed, 2010]. .	13
2.2	Diagrama de Pacotes.	16
3.1	Diagrama de classes.	18
3.2	Diagrama de sequência.	20
3.3	Diagrama de sequência.	21
3.4	Diagrama de comunicação.	22
3.5	Diagrama de máquina de estado.	23
3.6	Diagrama de atividades.	24
4.1	Diagrama de componentes.	28
4.2	Diagrama de implantação.	29
7.1	Tela do programa mostrando terminal executando.	53
7.2	Tela do programa mostrando gráfico (<i>Excel</i>) gerado a partir do <code>output.csv</code>	53
7.3	Tela do programa mostrando curva do modelo de Corey.	55
7.4	Tela do programa mostrando curva do Exemplo 14-5 do livro <i>Reservoir Engineering Handbook</i> de Tarek Ahmed [Ahmed, 2010] (com $\mu_o = 1.0$).	57
7.5	Curva do Exemplo 14-5 do livro <i>Reservoir Engineering Handbook</i> de Tarek Ahmed [Ahmed, 2010] (com $\mu_o = 0.5, \mu_o = 1.0, \mu_o = 5$ e $\mu_o = 10$).	58
7.6	Tela do programa mostrando captura de tela a janela do Gnuplot.	59
8.1	Página inicial.	62
8.2	Hierarquia de classes.	62
8.3	Página para <code>CalculadoraFluxoFracionario Class Reference</code>	63
8.4	Arquivo <code>CalculadoraFluxoFracionario.h</code>	63

Lista de Tabelas

1.1	Nome do Sistema/Produto.	8
1.2	Caso de uso 1.	10
4.1	Tabela classificação sistema.	31

Listagens

6.1	ICurvasPermeabilidade.h.	35
6.2	CurvasPermeabilidadeTabelada.h.	36
6.3	CurvasPermeabilidadeTabelada.cpp.	38
6.4	CurvasPermeabilidadeCorey.h.	40
6.5	CurvasPermeabilidadeCorey.cpp.	41
6.6	CalculadoraFluxoFracionario.h	43
6.7	CalculadoraFluxoFracionario.cpp.	44
6.8	Gnuplot.h.	46
6.9	Gnuplot.cpp.	46
6.10	Simulador.h.	47
6.11	Simulador.cpp	48
6.12	main.cpp.	50
7.1	Teste-01.in.	51
7.2	Teste-02.in.	54
7.3	Teste-Tarek-1cp.in.	56
8.1	Comando para instalar Compilador, Make e Gnuplot no MSYS2..	60
8.2	Instalação do Doxygen.	61
8.3	Gerando a documentação HTML.	61
A.1	Exemplo de arquivo de entrada para modelo tabelado.	65
A.2	Exemplo de arquivo de entrada para modelo Corey.	66
B.1	Comando para navegar até a pasta do projeto.	67
B.2	Limpando arquivos antigos.	67
B.3	Comando de compilação.	68
B.4	Executando o Teste 1.	68
B.5	Executando a Validação com Literatura.	68

Capítulo 1

Concepção

Apresenta-se neste capítulo do projeto de engenharia a concepção, a especificação do sistema a ser modelado e desenvolvido.

1.1 Metodologia

A Figura 1.1 apresenta a metodologia a ser utilizada no desenvolvimento do sistema.

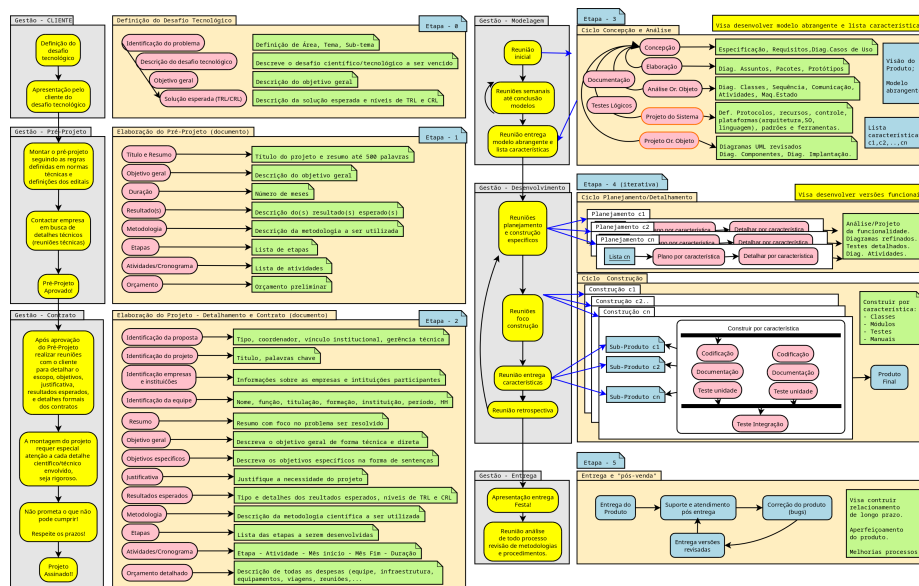


Figura 1.1: Metodologia utilizada no desenvolvimento do sistema.

O desenvolvimento do código-fonte e a estruturação dos documentos *LaTeX*

Tabela 1.1: Nome do Sistema/Produto.

Nome	Ferramenta para Geração da Curva de Fluxo Fracionário
Componentes principais	1. Módulo de Entrada (Leitor de arquivo de texto) 2. Módulo de Dados (Modelos de Permeabilidade) 3. Módulo de Cálculo (Equação de Buckley-Leverett) 4. Módulo de Saída (Gerador de .csv e interface com Gnuplot)
Missão	Missão, Calcular e visualizar a curva de fluxo fracionário (f_w vs. S_w) para auxiliar na análise de deslocamento bifásico em meios porosos.

foram auxiliados pelo uso de modelos de linguagem (IA) para otimização de sintaxe e depuração de erros de compilação. Todo o código gerado foi revisado, testado e validado pela autora.

1.2 Nome do Sistema/Produto

Abaixo a Tabela 1.1 com o nome do Sistema/Produto: Ferramenta para Geração da Curva de Fluxo Fracionário.

1.3 Especificação

A especificação desta solução de engenharia define uma ferramenta computacional analítica, de caráter didático e profissional, implementada como um executável de linha de comando. O sistema irá abstrair a complexidade do modelo matemático de fluxo fracionário, exigindo do usuário apenas um arquivo de configuração em texto simples. Este arquivo deverá conter os parâmetros reológicos dos fluidos (viscosidades) e os dados de permeabilidade relativa, que são suportados em dois formatos: (a) dados tabulares discretos (S_w, k_{rw}, k_{ro}), que serão processados via interpolação linear; ou (b) parâmetros para o modelo analítico de Corey [Corey, 1954]). O software processará esses dados, calculará a curva f_w vs S_w e gerará dois produtos: um arquivo de dados .csv para análise externa e uma visualização gráfica imediata, invocando o Gnuplot para plotar a curva "S" característica.

1.4 Requisitos

Apresenta-se nesta seção os requisitos funcionais e não funcionais.

Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

RF-01	O sistema deve ler um arquivo de entrada de texto contendo os parâmetros da simulação (viscosidades e dados de permeabilidade).
RF-02	O sistema deve permitir que os dados de permeabilidade relativa sejam fornecidos de duas formas: (a) uma tabela de pontos (S_w, k_{rw}, k_{ro}) ou (b) parâmetros para um modelo analítico (Correlações de Corey[Corey, 1954]).
RF-03	O sistema deve calcular os valores de fluxo fracionário (f_w) para um intervalo de saturação de água (S_w) de 0 a 1.
RF-04	O sistema deve gerar um arquivo de saída em formato de texto (ex: <code>.csv</code>) contendo os pares de dados (S_w, f_w) calculados.
RF-05	O sistema deve invocar um processo externo (Gnu-plot) para plotar automaticamente o gráfico de f_w vs. S_w .
RF-06	O sistema deve informar ao usuário em caso de erro na leitura do arquivo de entrada (ex: arquivo não encontrado, formato inválido).

Requisitos não funcionais

RNF-01	Os cálculos de permeabilidade relativa, quando tabulados, devem usar interpolação linear.
RNF-02	O programa deverá ser multi-plataforma, podendo ser executado em <i>Windows</i> e GNU/Linux.
RNF-03	O software deve ser desenvolvido na linguagem C++ utilizando o paradigma da Orientação a Objetos.
RNF-04	O software deve ser executável via terminal (linha de comando).

1.5 Casos de Uso

A Tabela 1.2 mostra a descrição de um caso de uso.

Tabela 1.2: Caso de uso 1.

Nome do caso de uso:	Executar Cálculo e Analisar Resultados
Resumo/descrição:	O usuário executa a ferramenta, que lê os dados de entrada, calcula a curva de fluxo fracionário e exibe o gráfico resultante.
Etapas:	1. Usuário executa o programa via terminal, fornecendo o nome do arquivo de entrada. 2. O sistema lê e valida o arquivo de entrada. 3. O sistema calcula a curva de f_w vs. S_w. 4. O sistema salva os resultados em um arquivo <code>.csv</code>. 5. O sistema invoca o Gnuplot para exibir o gráfico. 6. O usuário analisa o gráfico e fecha a janela.
Cenários alternativos:	2a. O arquivo de entrada não existe ou está mal formatado: O sistema exibe uma mensagem de erro clara e encerra. 5a. O Gnuplot não está instalado ou não foi encontrado no PATH do sistema: O sistema exibe um aviso, mas informa que o arquivo de dados <code>.csv</code> foi gerado com sucesso.

Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 1.2 mostra o ator principal, o Usuário (Engenheiro/Estudante), e as interações primárias que ele pode ter com o sistema.

Diagrama de caso de uso específico

O caso de uso Executar Cálculo é detalhado na Figura 1.3. Ele é composto por várias sub tarefas que são obrigatoriamente incluídas em sua execução.

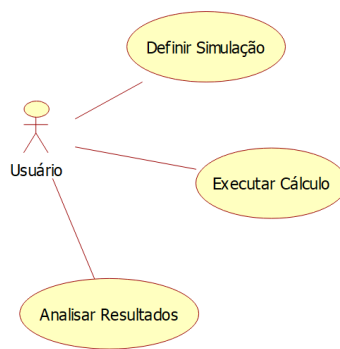


Figura 1.2: Diagrama de caso de uso – Caso de uso geral

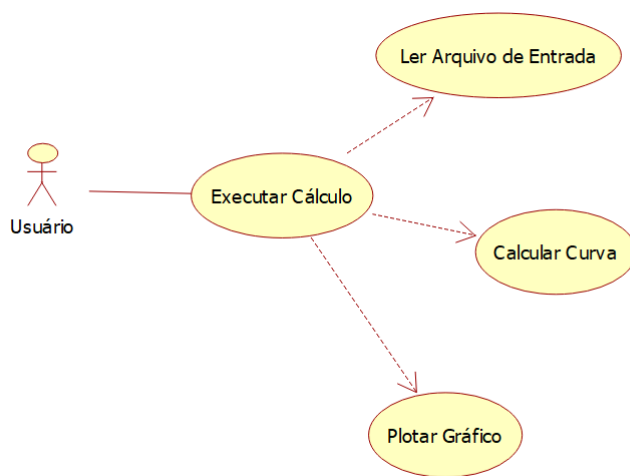


Figura 1.3: Diagrama de caso de uso específico – Executar Cálculo.

Capítulo 2

Elaboração

Este capítulo detalha o domínio do problema e a lógica de engenharia.

2.1 Análise de domínio

O domínio deste projeto situa-se na interseção da Engenharia de Reservatórios com a Modelagem Computacional. A seguir, detalha-se a análise deste domínio e as decisões de arquitetura de software que ela motivou.

- Justificativa da Plataforma Tecnológica: A linguagem C++ (padrão C++ 17) foi selecionada como base tecnológica. A decisão se justifica pelo seu comprovado desempenho computacional, controle de baixo nível de memória e vasta biblioteca padrão (STL). Essa escolha garante que o núcleo de cálculo seja eficiente e, futuramente, possa ser facilmente integrado como uma biblioteca em simuladores de reservatório mais complexos. O desenvolvimento visa a portabilidade entre *Windows* e GNU/Linux, utilizando ferramentas de compilação padrão (g++).
- Arquitetura de Dados: Optou-se por uma arquitetura *serverless* e sem banco de dados. A persistência dos dados de entrada é gerenciada pelo usuário via arquivos de texto `.txt` com formato baseado em palavras-chave. Esta decisão mitiga a complexidade de *deploy* e configuração, alinhando-se à natureza de uma ferramenta de análise rápida. O protocolo de saída (`.csv`) foi escolhido por sua universalidade, permitindo a interoperabilidade com qualquer software de planilha ou *script* de pós-processamento.
- Gerenciamento de Dependências: O projeto terá uma única dependência externa principal: o Gnuplot. Para manter o software leve e evitar a complexidade de linkagem de bibliotecas gráficas, o Gnuplot será invocado como um processo de sistema externo (`system()`). A classe Gnuplot do nosso projeto atuará como uma Facade, encapsulando essa chamada.

- Modelo de Controle: O fluxo de execução do software é puramente sequencial e síncrono (Leitura -> Cálculo -> Plotagem). Dada a baixa complexidade computacional do modelo 1D, não há necessidade de implementação de concorrência ou processamento paralelo, o que simplifica o design.

2.2 Formulação teórica

O núcleo deste projeto é a implementação da equação de fluxo fracionário (f_w) de Buckley-Leverett[Buckley and Leverett, 1942], conforme definido no documento do Desafio Tecnológico :

$$f_w = \frac{1}{1 + \frac{k_{ro}(S_w)}{\mu_o} \cdot \frac{\mu_w}{k_{rw}(S_w)}}$$

A curva, resultado da equação do fluxo fracionário possui uma característica de forma “S”, como está apresentado na Figura 2.1.

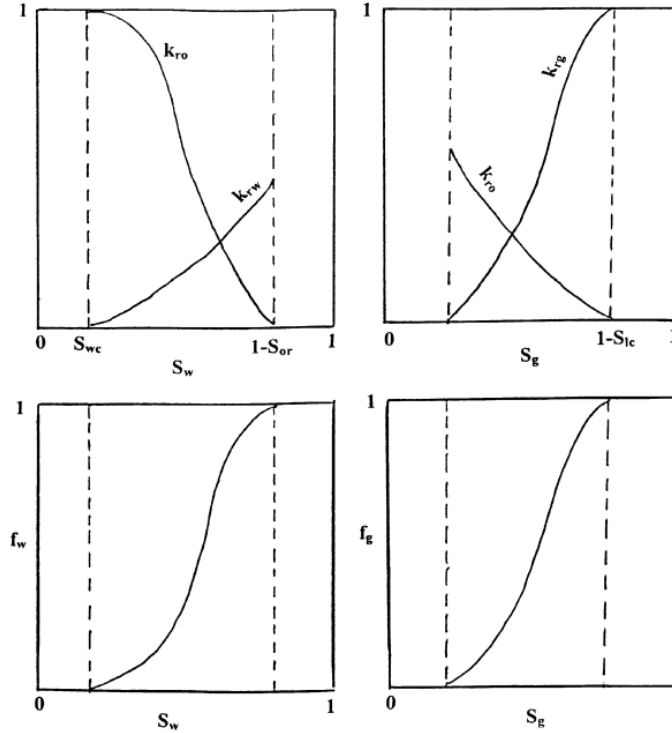


Figura 2.1: Curva de fluxo fracionário (f_w) característica, mostrando a sua forma de "S". Fonte: Adaptado de Ahmed, 2019 [Ahmed, 2010].

O desafio computacional reside em como obter os valores de $k_{ro}(S_w)$ e $k_{rw}(S_w)$ para qualquer valor de saturação de água S_w , uma vez que os dados de entrada podem vir de duas formas distintas: um modelo tabular ou um modelo analítico.

Implementação do Modelo Tabular (Interpolação Linear)

Quando os dados são fornecidos como uma tabela de pontos (lidos do arquivo de texto), o programa precisa calcular os valores de k_r para saturações que estão entre os pontos da tabela. Para isso, usaremos a Interpolação Linear.

- Algoritmo:
 1. O software lerá o arquivo de entrada e armazenará os N pontos de dados em três vetores (ou listas) paralelos: `vec_Sw[]`, `vec_Krw[]`, `vec_Kro[]`.
 2. Quando a função `calcularFw(Sw_desejada)` for chamada:
 3. O programa irá buscar no `vec_Sw[]` os dois valores, $S_{w,i}$ e $S_{w,i+1}$, que "cercam" o valor de S_w desejado (ou seja, $S_{w,i} \leq S_w \leq S_{w,i+1}$).
 4. A interpolação linear é aplicada para encontrar o k_{rw} e k_{ro} correspondentes:
 - $k_{rw} = k_{rw,i} + (S_{w,desejada} - S_{w,i}) \times \frac{k_{rw,i+1} - k_{rw,i}}{S_{w,i+1} - S_{w,i}}$
 - $k_{ro} = k_{ro,i} + (S_{w,desejada} - S_{w,i}) \times \frac{k_{ro,i+1} - k_{ro,i}}{S_{w,i+1} - S_{w,i}}$
 5. Estes valores interpolados de k_{rw} e k_{ro} são então usados na equação principal de f_w .
 6. Tratamento de Pontas: Se $S_{w,desejada}$ for menor que o primeiro valor da tabela, os valores do primeiro ponto ($k_{rw,0}, k_{ro,0}$) serão usados. Se for maior que o último ponto, os valores do último ponto ($k_{rw,N}, k_{ro,N}$) serão usados (sem extrapolação).

Implementação do Modelo Analítico (Correlações de Corey)

Quando o arquivo de entrada especifica o uso de um modelo analítico como o de Corey [Corey, 1954], o programa não usará uma tabela, mas sim um conjunto de parâmetros para calcular k_r diretamente.

- Algoritmo:
 1. O software lerá do arquivo de entrada os parâmetros do modelo de Corey:
 - S_{wir} : Saturação de água irreduzível.
 - S_{orw} : Saturação de óleo residual à injeção de água.

- $k_{rw,max}$ (ou k_{rwo}): Permeabilidade relativa à água na saturação de óleo residual.
 - $k_{ro,max}$ (ou k_{rocw}): Permeabilidade relativa ao óleo na saturação de água irreduzível.
 - n_w : Expoente de Corey para a fase água.
 - n_o : Expoente de Corey para a fase óleo.
2. Primeiro, calcula-se a saturação de água normalizada ($S_{w,norm}$) para qualquer S_w de entrada:
 - $S_{w,norm} = \frac{S_w - S_{wir}}{1 - S_{wir} - S_{orw}}$
 3. A $S_{w,norm}$ é limitada entre 0 e 1 (se $S_w < S_{wir}$, $S_{w,norm} = 0$; se $S_w > (1 - S_{orw})$, $S_{w,norm} = 1$).
 4. As permeabilidades relativas são então calculadas usando $S_{w,norm}$:
 - $k_{rw}(S_w) = k_{rw,max} \times (S_{w,norm})^{n_w}$
 - $k_{ro}(S_w) = k_{ro,max} \times (1 - S_{w,norm})^{n_o}$
 5. Estes valores calculados de k_{rw} e k_{ro} são então usados na equação principal de f_w .

A estrutura do software em C++ será projetada de forma que a calculadora principal de f_w não precise saber qual desses métodos está sendo usado, apenas que ela pode pedir os valores de k_r .

Para uma dedução mais detalhada destas equações e uma discussão aprofundada sobre as propriedades dos fluidos e modelos de permeabilidade, consulte as referências clássicas da engenharia de reservatórios, como Ahmed [Ahmed, 2010] e Lake [Lake et al., 2010].

2.3 Identificação de pacotes – assuntos

Identificamos os seguintes pacotes (assuntos) lógicos para a organização do sistema:

- **Pacote Core (Núcleo)**: Contém a classe principal com a lógica de engenharia, ou seja, a `CalculadoraFluxoFracionario`. Este pacote implementa a equação de Buckley-Leverett.
- **Pacote ModelosKr**: Contém a interface `ICurvasPermeabilidade` e suas implementações concretas (`CurvasPermeabilidadeTabelada` e `CurvasPermeabilidadeCorey`). Este pacote é responsável por fornecer os dados de permeabilidade relativa para o pacote Core.
- **Pacote IO (Entrada/Saída)**: Contém classes utilitárias responsáveis pela leitura e *parsing* do arquivo de entrada de texto e pela escrita do arquivo de saída `.csv`.

- Pacote **Visualizacao**: Contém a classe utilitária `Gnuplot`, responsável por formatar os dados de saída e invocar o processo externo do `Gnuplot` para plotagem.
- Pacote **Aplicacao**: Contém a classe `Simulador` (com a função `main()`), que orquestra todo o processo: lê os dados usando `IO`, instancia o `ModelosKr` correto, instancia o `Core` e, por fim, chama a `Visualizacao`.

2.4 Diagrama de pacotes – assuntos

O Diagrama de Pacotes, apresentado na Figura 2.2, ilustra as dependências entre os assuntos lógicos identificados.

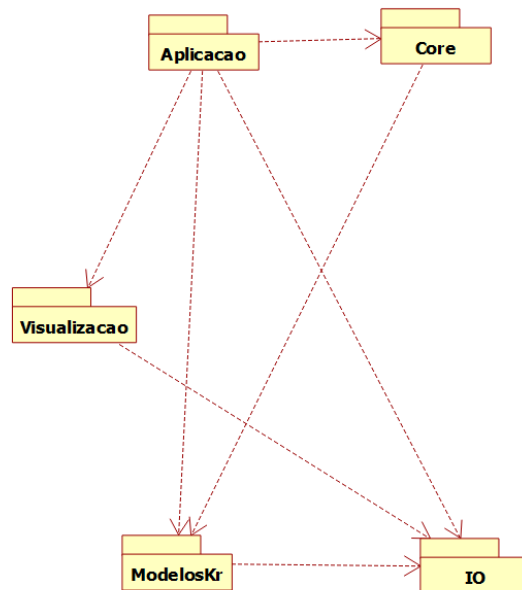


Figura 2.2: Diagrama de Pacotes.

Capítulo 3

AOO – Análise Orientada a Objeto

3.1 Diagramas de classes

O design estático do sistema, apresentado na Figura 3.1, é fundamentado em princípios S.O.L.I.D. de engenharia de software para garantir um código limpo, extensível e de fácil manutenção.

1. Princípio da Inversão de Dependência (D do S.O.L.I.D.): Este é o pilar central da arquitetura. O módulo de alto nível (`CalculadoraFluxoFracionario`) não depende diretamente dos módulos de baixo nível (`...Tabelada`, `...Corey`). Ambos dependem de uma abstração: a interface `ICurvasPermeabilidade`.
2. Padrão de Projeto *Strategy* (Estratégia): A interface `ICurvasPermeabilidade` implementa o padrão *Strategy*. Ela define um "contrato" para uma família de algoritmos (os modelos de k_r) e permite que eles sejam trocados dinamicamente. A Calculadora é configurada com uma dessas estratégias no momento da sua criação, através de Injeção de Dependência em seu construtor. Isso torna a calculadora completamente agnóstica aos detalhes de como o k_r é obtido.
3. Padrão de Projeto Facade (Fachada): A classe `Gnuplot` é uma Facade estática. Ela fornece uma interface unificada e simples (`plotarCurva()`) para um subsistema complexo (formatação de *scripts*, salvamento de arquivos temporários e chamada de processos do sistema operacional).

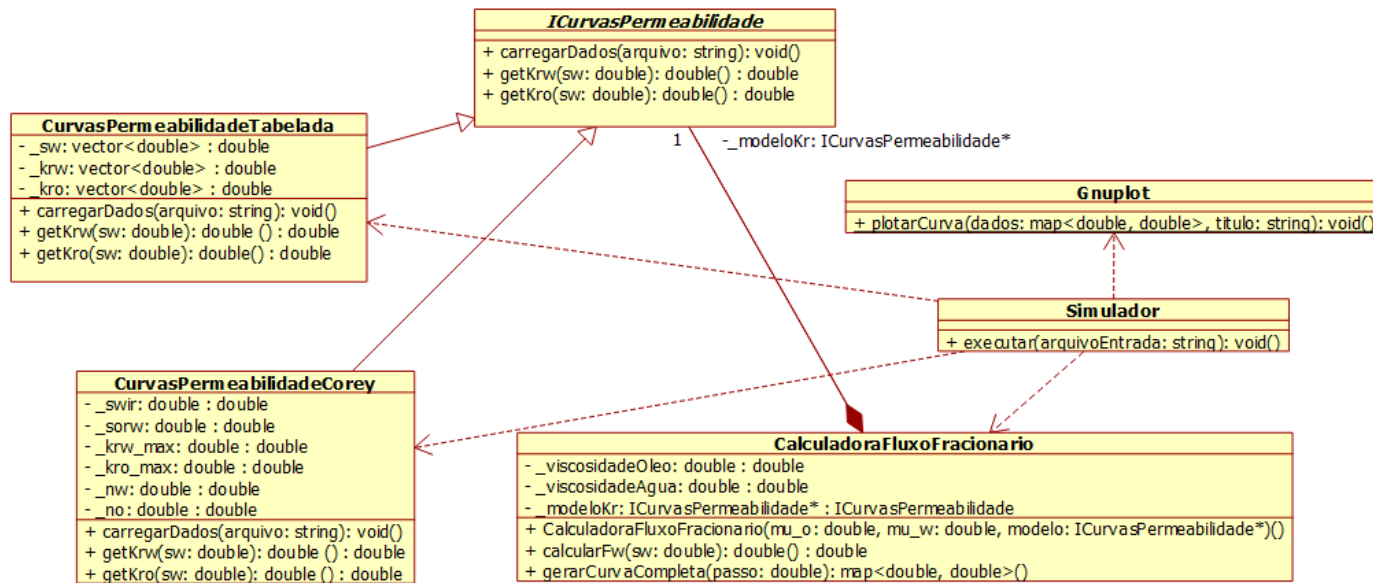


Figura 3.1: Diagrama de classes.

Dicionário de classes

- Classe **ICurvasPermeabilidade**: (Interface) Define o contrato de serviço para os modelos de permeabilidade relativa. Garante o polimorfismo e a Inversão de Dependência.
- Classe **CurvasPermeabilidadeTabelada**: (Implementação Concreta) Implementação da *Strategy* de k_r via dados tabulares. Encapsula a lógica de interpolação linear.
- Classe **CurvasPermeabilidadeCorey**: (Implementação Concreta) Implementação da *Strategy* de k_r via correlação analítica de Corey [Corey, 1954]. Encapsula a lógica matemática do modelo.
- Classe **CalculadoraFluxoFracionario**: (Domínio de Negócio) Classe principal que implementa o domínio central de negócio (a equação de Buckley-Leverett). Opera em um alto nível de abstração, dependendo apenas da interface **ICurvasPermeabilidade**.
- Classe **Gnuplot**: (Utilitário / Facade) Classe estática que encapsula toda a lógica de interação com o Gnuplot, simplificando a plotagem para o resto da aplicação.
- Classe **Simulador**: (Orquestrador / Ponto de Entrada) Atua como o orquestrador da aplicação. É responsável por "montar" o sistema (ler arquivos, instanciar os objetos corretos usando *new*) e injetar as dependências (`modeloKr` dentro da **Calculadora**). Contém o ponto de entrada `main()`.

3.2 Diagrama de sequência – eventos e mensagens

Diagrama de sequência geral

O diagrama de sequência geral, Figura 3.2, ilustra o fluxo principal de execução do software, correspondente ao caso de uso "Executar Cálculo e Analisar Resultados" com sucesso.

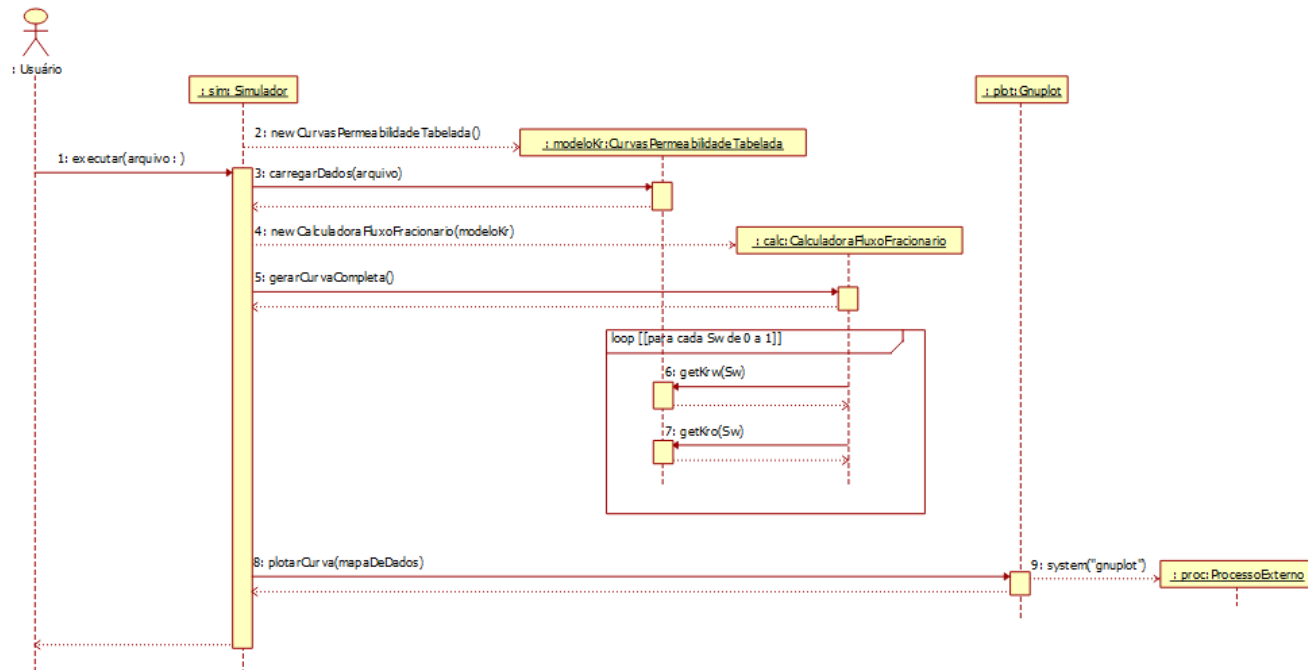


Figura 3.2: Diagrama de sequência.

Diagrama de sequência específico

O diagrama da Figura 3.3 detalha a atividade específica de `getKrw()` na classe `CurvasPermeabilidadeTabelada`, mostrando a lógica de interpolação.

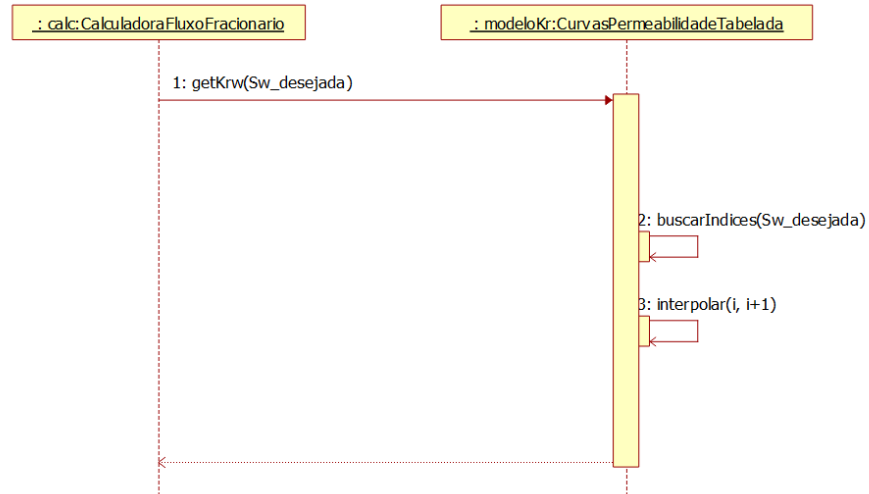


Figura 3.3: Diagrama de sequência.

3.3 Diagrama de comunicação – colaboração

O diagrama de comunicação da Figura 3.3 apresenta a mesma interação do diagrama de sequência geral, mas foca na colaboração e nas mensagens trocadas entre os objetos, em vez da ordem temporal.

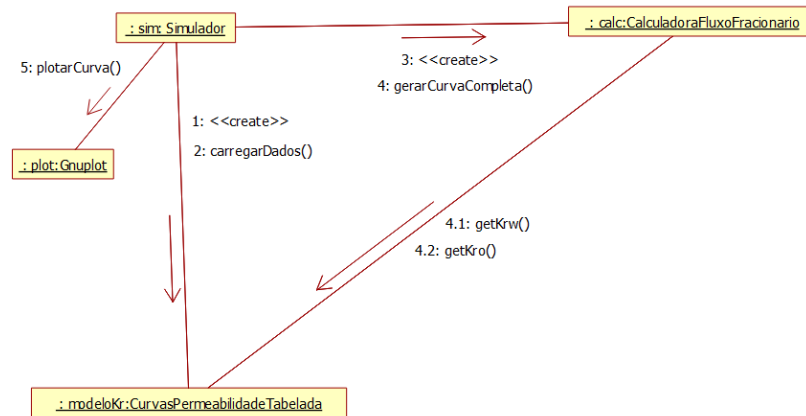


Figura 3.4: Diagrama de comunicação.

3.4 Diagrama de estado

O diagrama da Figura 3.5 modela os estados dinâmicos de um objeto *Simulador* ao longo de sua vida, desde a criação até a finalização da execução.

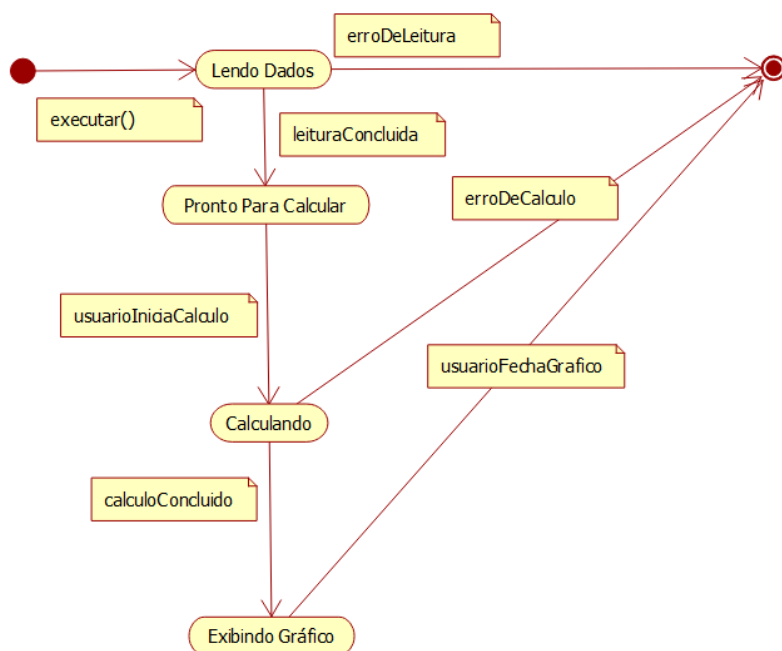


Figura 3.5: Diagrama de máquina de estado.

3.5 Diagrama de atividades

O diagrama de atividades da Figura 3.6 detalha o algoritmo do método `getKrw()` na classe `CurvasPermeabilidadeTabelada`, como descrito na Formulação Teórica. Este é um cálculo específico e crucial.

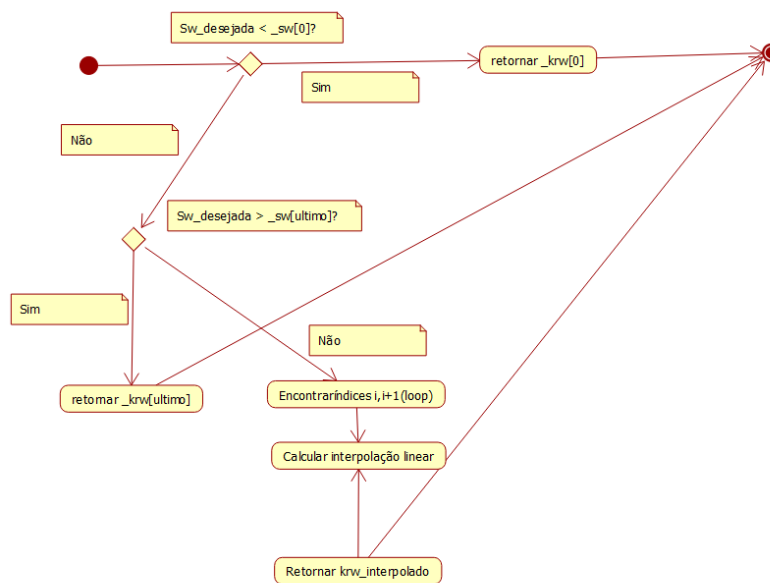


Figura 3.6: Diagrama de atividades.

Capítulo 4

Projeto

4.1 Projeto do sistema

1. Protocolos

- Comunicação Externa: O protocolo de comunicação com o usuário será via arquivos de texto. O formato do arquivo de entrada (detalhado no Apêndice A) é o protocolo principal. O sistema gerará um arquivo `.csv` (valores separados por vírgula) como protocolo de saída, garantindo fácil interoperabilidade.
- Comunicação Interna: A comunicação entre os objetos do sistema seguirá um padrão de passagem de mensagens síncronas (chamadas de método diretas).
- API: A API interna da biblioteca de cálculo será definida pela interface (classe base abstrata) `ICurvasPermeabilidade` e pela classe `CalculadoraFluxoFracionario`.

2. Recursos

- Banco de Dados: Não será utilizado um banco de dados. Para o escopo da aplicação (uma ferramenta de análise leve), um BD introduziria uma complexidade de implantação e configuração desnecessária.
- Armazenamento: A persistência dos dados de entrada é gerenciada pelo usuário através de arquivos `.txt`.

3. Controle

- Implementação: O controle do sistema será sequencial e baseado em procedimentos. A execução seguirá um fluxo linear (Ler -> Calcular -> Plotar).

- Concorrência: Não há necessidade de concorrência ou processamento paralelo (*multithreading*), pois o custo computacional do cálculo do fluxo fracionário 1D é baixo e a execução é instantânea.

4. Plataformas

- Hardware/SO: O software será desenvolvido em C++ padrão (C++17), garantindo alta portabilidade e compilação nativa em *Windows* e GNU/Linux.
- Bibliotecas Externas: A única dependência externa será o Gnuplot. Para manter o software leve, o Gnuplot não será linkado ao código; ele será invocado como um processo de sistema externo (`system()`).
- IDE: O desenvolvimento será feito utilizando um editor de texto (Kate) e compilação via *make* no terminal MSYS2.

5. Padrões de projeto

- Conforme detalhado no Capítulo 3, o design da arquitetura é centrado no uso dos padrões de projeto *Strategy* (para os modelos de permeabilidade) e *Facade* (para a classe Gnuplot).

4.2 Projeto orientado a objeto – POO

- Efeitos do projeto no modelo estrutural: A arquitetura definida na Análise (Cap. 3) é mantida. O uso do padrão *Strategy* garante que o Diagrama de Pacotes seja limpo: o pacote *Core* depende apenas da abstração em *ModelosKr*, não das implementações concretas.
- Efeitos do projeto no modelo dinâmico: Os diagramas de sequência e estado confirmam o fluxo de controle síncrono e sequencial. A chamada ao Gnuplot (`system()`) será bloqueante, garantindo que o programa só termine após o usuário fechar o gráfico.
- Efeitos do projeto nos atributos: Todos os atributos de classe serão privados (*private*), garantindo encapsulamento. A *CalculadoraFluxoFracionario* armazenará um ponteiro para a interface *ICurvasPermeabilidade*, minimizando o acoplamento.
- Efeitos do projeto nos métodos: Os métodos de cálculo (como `getKrw`, `calcularFw`) serão declarados como `const` sempre que possível, para garantir que não modifiquem o estado do objeto. O método `plotarCurva` da classe *Gnuplot* será `static`, pois não depende de um estado de instância.
- Efeitos do projeto nas heranças: A herança será usada exclusivamente para implementar interfaces (herança de tipo), não para reutilização de código. Isso segue o princípio de "Composição sobre Herança".

- Efeitos do projeto nas associações: A associação principal é a de uso (agregação), onde a Calculadora usa a interface `ICurvasPermeabilidade` que lhe é injetada. O Simulador gerencia o ciclo de vida (criação *new* e destruição *delete*) dos objetos.
- Efeitos do projeto nas otimizações: Nenhuma otimização de baixo nível (ex: paralelismo) é necessária. A otimização do projeto está no design de O.O., que evita recálculos e promove baixo acoplamento.

4.3 Diagrama de componentes

O Diagrama de Componentes (Figura 4.1) detalha as dependências de compilação e ligação do software. O componente executável final, `fw_calc` , depende da compilação e linkagem de múltiplos componentes de código-fonte (`.cpp`). Cada componente `.cpp` (ex: `Simulador.cpp`, `Calculadora.cpp`) depende do seu respectivo arquivo de cabeçalho (`.h`) e das abstrações das quais depende (ex: `Calculadora.cpp` depende de `ICurvasPermeabilidade.h`). O componente `Gnuplot.cpp` não depende do executável do Gnuplot em tempo de compilação, mas o executável `fw_calc` tem uma dependência em tempo de execução com o Gnuplot.

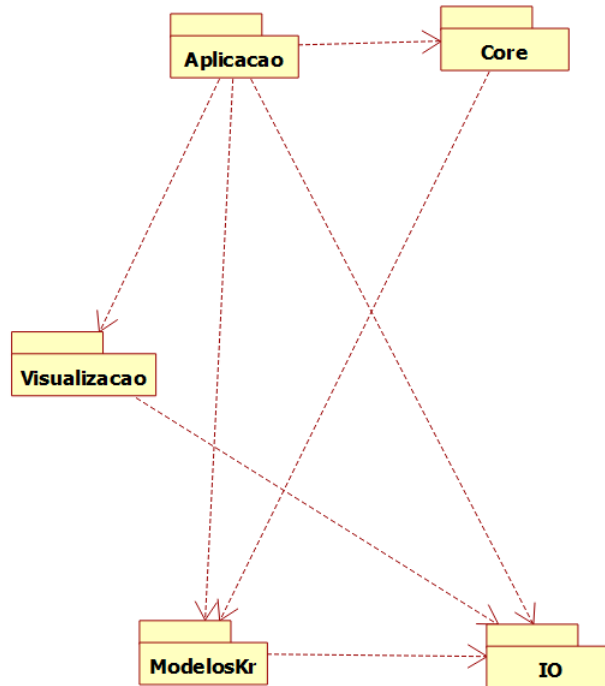


Figura 4.1: Diagrama de componentes.

4.4 Diagrama de implantação

O Diagrama de Implantação (Figura 4.2) para este projeto é simples, pois não se trata de um sistema distribuído. Haverá um único nó principal: <<device>> Computador Pessoal. Este nó de hardware hospeda o sistema operacional (ex: *Windows* 10/11 ou GNU/Linux). Dentro deste S.O., residirão os dois artefatos de software necessários para a execução:

1. [fw_calc] : O nosso artefato executável compilado.
2. [gnuplot]: O artefato executável da dependência externa. O [fw_calc] possui uma dependência de execução (do tipo <<call>>) com o [gnuplot].

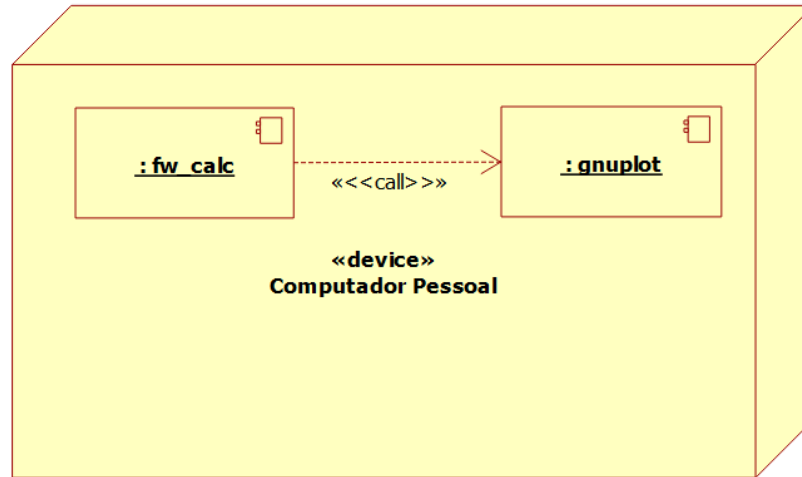


Figura 4.2: Diagrama de implantação.

Lista de características <<features>>

No final do ciclo de concepção e análise chegamos a uma lista de características <<features>> que teremos de implementar.

Após a análises desenvolvidas e considerando o requisito de que este material deve ter um formato didático, chegamos a seguinte lista:

- v0.1
 - Implementar classes Simulador, CalculadoraFluxoFracionario.
 - Implementar ICurvasPermeabilidade e CurvasPermeabilidadeTabelada.
 - Implementar leitor de arquivo de entrada (parser) para o modelo tabelado e viscosidades.
 - Implementar cálculo da curva e salvamento em arquivo `output.csv`.
 - Testes: Validar o `.csv` de saída contra um cálculo manual em planilha.
- v0.2
 - Implementar classe CurvasPermeabilidadeCorey [Corey, 1954].
 - Atualizar o parser do arquivo de entrada para ler os parâmetros do modelo Corey .
 - Testes: Validar o `.csv` de saída para o modelo Corey.

- v0.3
 - Implementar classe Gnuplot .
 - Integrar a chamada do Gnuplot no Simulador para plotagem automática.
 - Testes: Validar a exibição do gráfico em *Windows* e *Linux*.

Tabela classificação sistema

A Tabela 4.1 a seguir é utilizada para classificação do sistema desenvolvido.

Tabela 4.1: Tabela classificação sistema.

Licença:	☒ livre GPL-v3 ☐ proprietária
Engenharia de software:	☐ tradicional ☒ ágil ☐ outras
Paradigma de programação:	☐ estruturada ☒ orientado a objeto - POO ☐ funcional
Modelagem UML:	☒ básica ☒ intermediária ☐ avançada
Algoritmos:	☒ alto nível ☐ baixo nível
	implementação: ☐ recursivo ou ☒ iterativo; ☒ determinístico ou ☐ não-determinístico; ☐ exato ou ☒ aproximado
	concorrências: ☒ serial ☐ concorrente ☐ paralelo
	paradigma: ☒ dividir para conquistar ☐ programação linear ☐ transformação/ redução ☐ busca e enumeração ☐ heurístico e probabilístico ☐ baseados em pilhas
Software:	☐ de base ☒ aplicados ☐ de cunho geral ☒ específicos para determinada área ☒ educativo ☒ científico
	instruções: ☒ alto nível ☐ baixo nível
	otimização: ☒ serial não otimizado ☐ serial otimizado ☐ concorrente ☐ paralelo ☐ vetorial
	interface do usuário: ☐ kernel numérico ☒ linha de comando ☐ modo texto ☒ híbrida (texto e saídas gráficas) ☐ modo gráfico (ex: Qt) ☐ navegador
Recursos de C++:	☒ C++ básico (FCC): variáveis padrões da linguagem, estruturas de controle e repetição, estruturas de dados, struct, classes(objetos, atributos, métodos), funções; entrada e saída de dados (<i>streams</i>), funções de cmath
	☒ C++ intermediário: funções lambda. Ponteiros, referências, herança, herança múltipla, polimorfismo, sobrecarga de funções e de operadores, tipos genéricos (templates), <i>smart pointers</i> . Diretrizes de pré-processador, classes de armazenamento e modificadores de acesso. Estruturas de dados: enum, uniões. Bibliotecas: entrada e saída acesso com arquivos de disco, redirecionamento. Bibliotecas: <i>filesystem</i>
	☒ C++ intermediário 2: A biblioteca de gabaritos de C++ (a STL), containers, iteradores, objetos funções e funções genéricas. Noções de processamento paralelo (múltiplas threads, uso de <i>thread</i> , <i>join</i> e <i>mutex</i>). Bibliotecas: <i>random</i> , <i>threads</i>
	☐ C++ avançado: Conversão de tipos do usuário, especializações de templates, excessões. Cluster de computadores, processamento paralelo e concorrente, múltiplos processos (pipes, memória compartilhada, sinais). Bibliotecas: <i>expressões regulares</i> , <i>múltiplos processos</i>
Bibliotecas de C++:	☒ Entrada e saída de dados (<i>streams</i>) ☒ cmath ☒ <i>filesystem</i> ☐ <i>random</i> ☐ <i>threads</i> ☐ <i>expressões regulares</i> ☐ <i>múltiplos processos</i>
Bibliotecas externas:	☒ CGnuplot ☐ QCustomPlot ☐ Qt diálogos ☐ QT Janelas/menus/BT
Ferramentas auxiliares:	Montador: ☒ make ☐ cmake ☐ qmake
IDE:	☒ Editor simples: kate/gedit/emacs ☐ kdevelop ☐ QT-Creator ☐
SCV:	☐ cvs ☐ svn ☒ git
Disciplinas correlacionadas	☐ estatística ☒ cálculo numérico ☐ modelamento numérico ☐ análise e processamento de imagens

Capítulo 5

Ciclos de Planejamento/Detalhamento

Apresenta-se neste capítulo os ciclos de planejamento/detalhamento para as diferentes versões desenvolvidas. O projeto será dividido em quatro ciclos de desenvolvimento e validação.

5.1 Ciclo 1 (Versão 0.1) - MVP: Núcleo de Cálculo e Modelo Tabular

Este ciclo inicial foca em construir o Produto Mínimo Viável (MVP). O objetivo é validar a arquitetura central e a lógica de engenharia principal (a equação de Buckley-Leverett). Ao final deste ciclo, teremos um software de linha de comando que lê um arquivo `.txt` de um modelo tabelado, calcula a curva de fluxo fracionário e salva um arquivo `.csv` com o resultado.

- Funcionalidades (**features**) implementadas:
 - Implementação da `main()` (no `Simulador.cpp`) para orquestrar o processo.
 - Implementação do parser de arquivo de entrada (no pacote IO) para ler `VISC_OLEO`, `VISC_AGUA` e os dados do `MODELO_KR TABELADO`.
 - Implementação da classe `CalculadoraFluxoFracionario`.
 - Implementação da interface `ICurvasPermeabilidade`.
 - Implementação da classe `CurvasPermeabilidadeTabelada`, incluindo o algoritmo de interpolação linear.
- Teste de Validação do Ciclo 1:
 - O software deve ser executado com o arquivo de entrada `Teste-01.in`. O resultado `output.csv` gerado deve ser comparado, via Excel, com

uma planilha de cálculo manual usando os mesmos dados de entrada. Os resultados devem ser idênticos.

5.2 Ciclo 2 (Versão 0.2) - Extensão: Modelo Analítico Corey

Com o núcleo de cálculo validado, este ciclo foca em demonstrar a extensibilidade do software, conforme projetado no Diagrama de Classes (padrão *Strategy*). O objetivo é adicionar uma nova forma de cálculo (Correlação de Corey [Corey, 1954]) sem alterar o núcleo (`CalculadoraFluxoFracionario`).

- Funcionalidades (**features**) implementadas:
 - Implementação da classe `CurvasPermeabilidadeCorey`, herdando de `ICurvasPermeabilidade`.
 - Implementação do algoritmo matemático das equações de Corey. Atualização do parser de IO para reconhecer a palavra-chave: `MODELO_KR_COREY`, e ler os seis parâmetros de entrada.
 - Modificação no Simulador para instanciar o objeto `CurvasPermeabilidadeCorey` quando a palavra-chave for encontrada.
- Teste de Validação do Ciclo 2:
 - O software será executado com o arquivo `Teste-02.in`. Como o modelo de Corey é analítico, os resultados no `output.csv` serão validados comparando-os com uma planilha que implementa as mesmas fórmulas de Corey, garantindo a precisão matemática da implementação.

5.3 Ciclo 3 (Versão 0.3) - Validação Científica: Tarek Ahmed

Este ciclo é dedicado exclusivamente à validação científica da ferramenta. O objetivo não é adicionar código novo, mas sim provar que o modelo numérico (tabelado) consegue reproduzir resultados clássicos da literatura de engenharia de petróleo.

- Atividades Realizadas:
 - Levantamento de dados do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [Ahmed, 2010].
 - Criação do arquivo de entrada específico `Teste-Tarek-1cp.in` contendo os dados de permeabilidade e viscosidade extraídos do livro.
 - Execução e análise comparativa.

- Teste de Validação do Ciclo 3:
 - O software será executado com o Teste-Tarek-1cp.in. Os resultados no `output.csv` serão validados ponto a ponto contra a tabela de resultados publicada na página 950 do livro de Tarek Ahmed [Ahmed, 2010], como detalhado no Capítulo 7.

5.4 Ciclo 4 (Versão 0.4) - Apresentação: Integração Gnuplot

Este ciclo final foca na camada de apresentação e na usabilidade da ferramenta. O objetivo é implementar a *Facade* Gnuplot para fornecer *feedback* visual imediato ao usuário, completando o requisito RF-05.

- Funcionalidades (**features**) implementadas:
 - Implementação da classe Gnuplot como uma classe estática utilitária.
 - Implementação do método `Gnuplot::plotarCurva()`, que cria os arquivos temporários de dados e *script*, e invoca o `system()` para chamar o Gnuplot.
 - Integração da chamada `Gnuplot::plotarCurva()` no Simulador.
- Teste de Validação do Ciclo 4:
 - O software foi executado com sucesso utilizando os arquivos de teste anteriores, e a janela do Gnuplot com o gráfico foi exibida automaticamente em todos os casos, validando a integração.

Capítulo 6

Ciclos Construção - Implementação

6.1 Código fonte

Apresenta-se a seguir o conjunto de classes (arquivos `.h` e `.cpp`) que compõem a arquitetura do software, conforme o Diagrama de Classes (3.1).

- Interface

Listagem 6.1: `ICurvasPermeabilidade.h`.

```
1  #ifndef ICURVASPERMEABILIDADE_H
2  #define ICURVASPERMEABILIDADE_H
3
4  #include <string>
5
6  /**
7   * @class ICurvasPermeabilidade
8   * @brief Interface (classe base abstrata) para os modelos de
9   *       permeabilidade relativa.
10  *
11  * Esta classe define o "contrato" que todos os modelos de Kr (
12  *   Tabelado, Corey, etc.)
13  *   devem seguir. Ela é a base do Padrão de Projeto Strategy,
14  *   permitindo que a
15  *   Calculadora opere em um alto nível de abstração (Inversão de
16  *   Dependência).
17  */
18  class ICurvasPermeabilidade {
19  public:
20     /**
21      * @brief Destrutor virtual padrão, essencial para classes
22      *       base polimórficas.
23      */
24     virtual ~ICurvasPermeabilidade() {}
25
26     /**
```

```

22      * @brief Método virtual puro para carregar dados de um
        arquivo.
23      *
24      * Cada classe filha deve implementar este método para ler
        seus
25      * parâmetros específicos do arquivo de entrada (ex: ler a
        tabela ou ler os parâmetros de Corey).
26      * @param arquivo O caminho (path) para o arquivo de
        configuração .txt.
27      */
28      virtual void carregarDados(const std::string& arquivo) = 0;
29
30      /**
31      * @brief Obtém a permeabilidade relativa da água (Krw).
32      * @param sw A saturação de água (Sw) para a qual o Krw será
        calculado.
33      * @return O valor de Krw (entre 0 e 1).
34      */
35      virtual double getKrw(double sw) const = 0;
36
37      /**
38      * @brief Obtém a permeabilidade relativa do óleo (Kro).
39      * @param sw A saturação de água (Sw) para a qual o Kro será
        calculado.
40      * @return O valor de Kro (entre 0 e 1).
41      */
42      virtual double getKro(double sw) const = 0;
43  };
44
45  #endif

```

Apresenta-se na Listagem 6.1 o arquivo de cabeçalho da interface

ICurvasPermeabilidade. Esta classe base abstrata define o contrato para todos os modelos de permeabilidade, sendo o pilar do Padrão de Projeto *Strategy* (Nota: Esta classe é puramente virtual, portanto não possui um arquivo .cpp).

- Classe **CurvasPermeabilidadeTabelada**

Apresenta-se na listagem 6.2 o arquivo de cabeçalho da classe

CurvasPermeabilidadeTabelada, que implementa a interface para dados tabulares.

Listagem 6.2: CurvasPermeabilidadeTabelada.h.

```

1  #ifndef CURVASPERMEABILIDADETABELADA_H
2  #define CURVASPERMEABILIDADETABELADA_H
3
4  #include "ICurvasPermeabilidade.h"
5  #include <vector>
6  #include <string> // Incluído para std::string
7
8  /**
9   * @class CurvasPermeabilidadeTabelada
10  * @brief Implementação concreta da interface
        ICurvasPermeabilidade para dados tabulados.
11  *

```

```

12  * Esta classe lê uma tabela de Sw, Krw e Kro de um arquivo de
    entrada e usa
13  * interpolação linear para calcular valores intermediários.
14  */
15  class CurvasPermeabilidadeTabelada : public ICurvasPermeabilidade
    {
16  private:
17      /// Vetor com os valores de Saturação de Água da tabela.
18      std::vector<double> _sw;
19
20      /// Vetor com os valores de Krw da tabela.
21      std::vector<double> _krw;
22
23      /// Vetor com os valores de Kro da tabela.
24      std::vector<double> _kro;
25
26      /**
27       * @brief Algoritmo de Interpolação Linear (e extrapolação de
        ponta).
28       * Este é o algoritmo detalhado no Diagrama de Atividades.
29       * @param x_desejado A saturação (Sw) que queremos.
30       * @param vec_x O vetor de Saturações da tabela (_sw).
31       * @param vec_y O vetor de Krw ou Kro correspondente.
32       * @return O valor de y (Kr) interpolado.
33       */
34      double interpolar(double x_desejado, const std::vector<double>
        &vec_x, const std::vector<double>&vec_y) const;
35
36  public:
37      /**
38       * @brief Carrega os dados da tabela (bloco DADOS_KR_INICIO)
        do arquivo.
39       * @param arquivo O caminho para o arquivo de configuração .
        txt.
40       */
41      void carregarDados(const std::string& arquivo) override;
42
43      /**
44       * @brief Obtém o Krw, usando interpolação se necessário.
45       * @param sw A saturação de água.
46       * @return O valor de Krw interpolado.
47       */
48      double getKrw(double sw) const override;
49
50      /**
51       * @brief Obtém o Kro, usando interpolação se necessário.
52       * @param sw A saturação de água.
53       * @return O valor de Kro interpolado.
54       */
55      double getKro(double sw) const override;
56  };
57  #endif

```

Apresenta-se na listagem 6.3 a implementação da classe, incluindo o algoritmo de interpolação linear.

Listagem 6.3: CurvasPermeabilidadeTabelada.cpp.

```

1  #include "CurvasPermeabilidadeTabelada.h"
2  #include <iostream>
3  #include <fstream>
4  #include <sstream>
5  #include <stdexcept>
6
7  /**
8   * @brief Carrega os dados tabelados (Sw, Krw, Kro) do arquivo de
9     entrada.
10   * @param arquivo O caminho para o arquivo de configuração .txt.
11   */
12 void CurvasPermeabilidadeTabelada::carregarDados(const std::string
13     & arquivo) {
14     std::ifstream arq(arquivo);
15     if (!arq.is_open()) {
16         throw std::runtime_error("Erro(Tabelado):_Nao_foi_
17             possivel_abrir_o_arquivo:_ " + arquivo);
18     }
19
20     std::string linha;
21     bool lendoDados = false;
22
23     while (std::getline(arq, linha)) {
24         if (linha.empty() || linha[0] == '#') continue;
25
26         std::stringstream ss(linha);
27         std::string palavraChave;
28         ss >> palavraChave; // Lê a primeira "palavra"
29
30         if (palavraChave == "DADOS_KR_INICIO") {
31             lendoDados = true;
32             continue;
33         }
34
35         if (palavraChave == "FIM_DADOS") {
36             lendoDados = false;
37             break;
38         }
39
40         if (lendoDados) {
41             // Se estamos lendo, a "palavraChave" é na verdade o
42             // valor do Sw
43             std::stringstream ss_linha(linha);
44             double sw, krw, kro;
45             ss_linha >> sw >> krw >> kro;
46
47             // Adiciona os valores aos vetores da classe
48             _sw.push_back(sw);
49             _krw.push_back(krw);
50             _kro.push_back(kro);
51         }
52     }
53
54     if (_sw.empty()) {
55         throw std::runtime_error("Erro:_Nenhum_dado_de_
56             permeabilidade_encontrado_(bloco_DADOS_KR_INICIO...)

```

```

        FIM_DADOS)_no_arquivo.");
52     }
53     std::cout << "DEBUG:_" << _sw.size() << "_pontos_de_Kr_
        tabelados_foram_carregados.\n";
54 }
55
56 /**
57  * @brief Calcula Krw para uma Sw, usando interpolação linear se
        necessário.
58  * @param sw Saturação de água.
59  * @return Valor de Krw interpolado.
60  */
61 double CurvasPermeabilidadeTabelada::getKrw(double sw) const {
62     return interpolar(sw, _sw, _krw); // Chama a função de
        interpolação
63 }
64
65 /**
66  * @brief Calcula Kro para uma Sw, usando interpolação linear se
        necessário.
67  * @param sw Saturação de água.
68  * @return Valor de Kro interpolado.
69  */
70 double CurvasPermeabilidadeTabelada::getKro(double sw) const {
71     return interpolar(sw, _sw, _kro); // Chama a função de
        interpolação
72 }
73
74 /**
75  * @brief Algoritmo de Interpolação Linear (e extrapolação de
        ponta).
76  * Este é o algoritmo detalhado no Diagrama de Atividades.
77  * @param x_desejado A saturação (Sw) que queremos.
78  * @param vec_x O vetor de Saturações da tabela.
79  * @param vec_y O vetor de Krw ou Kro correspondente.
80  * @return O valor de y (Kr) interpolado.
81  */
82 double CurvasPermeabilidadeTabelada::interpolar(double x_desejado,
        const std::vector<double>& vec_x, const std::vector<double>&
        vec_y) const {
83
84     // Caso 1: Extrapolação (abaixo do limite inferior)
85     if (x_desejado <= vec_x.front()) {
86         return vec_y.front();
87     }
88
89     // Caso 2: Extrapolação (acima do limite superior)
90     if (x_desejado >= vec_x.back()) {
91         return vec_y.back();
92     }
93
94     // Caso 3: Interpolação (procurar o intervalo)
95     for (size_t i = 0; i < vec_x.size() - 1; ++i) {
96
97         // Achamos o intervalo: vec_x[i] <= x_desejado <= vec_x[i
            +1]
98         if (x_desejado >= vec_x[i] && x_desejado <= vec_x[i+1]) {

```

```

99
100         // Pontos do intervalo
101         double x0 = vec_x[i];
102         double y0 = vec_y[i];
103         double x1 = vec_x[i+1];
104         double y1 = vec_y[i+1];
105
106         // Evita divisão por zero se os pontos Sw forem
107         // idênticos
108         if (x1 == x0) {
109             return y0;
110         }
111         // Fórmula da Interpolação Linear:
112         //  $y = y0 + (x - x0) * (y1 - y0) / (x1 - x0)$ 
113         return y0 + (x_desejado - x0) * (y1 - y0) / (x1 - x0);
114     }
115 }
116
117 // Se algo der muito errado (não deveria acontecer)
118 return vec_y.back();
119 }

```

- Classe `CurvasPermeabilidadeCorey`

Apresenta-se na listagem 6.4 o arquivo de cabeçalho da classe

`CurvasPermeabilidadeCorey`, que implementa a interface para o modelo analítico de Corey [Corey, 1954].

Listagem 6.4: `CurvasPermeabilidadeCorey.h`.

```

1 #ifndef CURVASPERMEABILIDADECOREY_H
2 #define CURVASPERMEABILIDADECOREY_H
3
4 #include "ICurvasPermeabilidade.h"
5 #include <string> // Incluído para std::string
6
7 /**
8  * @class CurvasPermeabilidadeCorey
9  * @brief Implementação concreta da interface
10         ICurvasPermeabilidade para o modelo de Corey.
11  *
12  * Esta classe lê os 6 parâmetros do modelo de Corey do arquivo de
13         entrada
14  * e calcula  $K_{rw}/K_{ro}$  analiticamente usando as fórmulas de Corey.
15  */
16 #class CurvasPermeabilidadeCorey : public ICurvasPermeabilidade {
17 private:
18     /// Saturação de água irreduzível ( $S_{wir}$ )
19     double _swir;
20
21     /// Saturação de óleo residual ( $S_{orw}$ )
22     double _sorw;
23
24     /// Permeabilidade relativa máxima da água (no  $S_{orw}$ )
25     double _krw_max;
26

```



```

25     /// Permeabilidade relativa máxima do óleo (no Swir)
26     double _kro_max;
27
28     /// Expoente de Corey para a água (nw)
29     double _nw;
30
31     /// Expoente de Corey para o óleo (no)
32     double _no;
33
34 public:
35     /**
36      * @brief Carrega os 6 parâmetros do modelo Corey do arquivo.
37      * @param arquivo O caminho para o arquivo de configuração .
38      *      txt.
39      */
39     void carregarDados(const std::string& arquivo) override;
40
41     /**
42      * @brief Calcula o Krw usando a fórmula de Corey.
43      * @param sw A saturação de água.
44      * @return O valor de Krw analítico.
45      */
46     double getKrw(double sw) const override;
47
48     /**
49      * @brief Calcula o Kro usando a fórmula de Corey.
50      * @param sw A saturação de água.
51      * @return O valor de Kro analítico.
52      */
53     double getKro(double sw) const override;
54 };
55
56 #endif

```

Apresenta-se na listagem 6.5 a implementação da classe, contendo a formulação matemática do modelo.

Listagem 6.5: CurvasPermeabilidadeCorey.cpp.

```

1 #include "CurvasPermeabilidadeCorey.h"
2 #include <iostream>
3 #include <fstream>
4 #include <sstream>
5 #include <stdexcept>
6 #include <cmath>      // Para std::pow
7 #include <algorithm> // Para std::max e std::min
8
9 /**
10  * @brief Carrega os parâmetros do modelo de Corey do arquivo de
11  *      entrada.
12  * @param arquivo O caminho para o arquivo de configuração .txt.
13  */
13 void CurvasPermeabilidadeCorey::carregarDados(const std::string&
14     arquivo) {
15     // Seta valores padrão inválidos para checagem
16     _swir = _sow = _krw_max = _kro_max = _nw = _no = -1.0;
17     std::ifstream arq(arquivo);

```

```

18     if (!arq.is_open()) {
19         throw std::runtime_error("Erro (Corey): Não foi possível
        abrir o arquivo: " + arquivo);
20     }
21
22     std::string linha;
23     std::string palavraChave;
24
25     while (std::getline(arq, linha)) {
26         if (linha.empty() || linha[0] == '#') continue;
27
28         std::stringstream ss(linha);
29         ss >> palavraChave;
30
31         // Lê e armazena cada parâmetro
32         if (palavraChave == "COREY_SWIR") ss >> _swir;
33         else if (palavraChave == "COREY_SORW") ss >> _sorw;
34         else if (palavraChave == "COREY_KRW_MAX") ss >> _krw_max;
35         else if (palavraChave == "COREY_KRO_MAX") ss >> _kro_max;
36         else if (palavraChave == "COREY_NW") ss >> _nw;
37         else if (palavraChave == "COREY_NO") ss >> _no;
38     }
39
40     // Validação simples
41     if (_swir < 0 || _sorw < 0 || _krw_max < 0 || _kro_max < 0 ||
        _nw < 0 || _no < 0) {
42         throw std::runtime_error("Erro: Um ou mais parâmetros do
        modelo Corey não foram carregados corretamente.");
43     }
44     std::cout << "DEBUG: Parametros de Corey carregados com
        sucesso.\n";
45 }
46
47 /**
48  * @brief Calcula a Saturação Normalizada (Sw_norm).
49  * Privado, apenas para uso interno desta classe.
50  */
51 double calcularSwNorm(double sw, double swir, double sorw) {
52     // Fórmula: Sw_norm = (Sw - Swir) / (1 - Swir - Sorw)
53     double sw_norm = (sw - swir) / (1.0 - swir - sorw);
54
55     // Limita o resultado entre 0 e 1 (clamp)
56     return std::max(0.0, std::min(1.0, sw_norm));
57 }
58
59 /**
60  * @brief Calcula Krw usando a fórmula de Corey.
61  * @param sw Saturação de água.
62  * @return Valor de Krw.
63  */
64 double CurvasPermeabilidadeCorey::getKrw(double sw) const {
65     // 1. Calcular Sw_norm
66     double sw_norm = calcularSwNorm(sw, _swir, _sorw);
67
68     // 2. Fórmula de Corey para Krw: krw = krw_max * (Sw_norm ^ nw)
69     return _krw_max * std::pow(sw_norm, _nw);

```

```

70 }
71
72 /**
73  * @brief Calcula Kro usando a fórmula de Corey.
74  * @param sw Saturação de água.
75  * @return Valor de Kro.
76  */
77 double CurvasPermeabilidadeCorey::getKro(double sw) const {
78     // 1. Calcular Sw_norm
79     double sw_norm = calcularSwNorm(sw, _swir, _sorw);
80
81     // 2. Fórmula de Corey para Kro: kro = kro_max * ((1 - Sw_norm)
82     //      ^ no)
83     return _kro_max * std::pow(1.0 - sw_norm, _no);
84 }

```

- Classe CalculadoraFluxoFracionario

Apresenta-se na listagem 6.6 o arquivo de cabeçalho da classe

CalculadoraFluxoFracionario. Esta é a classe central do domínio de negócio, responsável por implementar a equação de Buckley-Leverett [Buckley and Leverett, 1942].

Listagem 6.6: CalculadoraFluxoFracionario.h

```

1  #ifndef CALCULADORAFLUXOFRACIONARIO_H
2  #define CALCULADORAFLUXOFRACIONARIO_H
3
4  #include "ICurvasPermeabilidade.h"
5  #include <map>
6  #include <string> // Incluído para std::string
7
8  /**
9   * @class CalculadoraFluxoFracionario
10  * @brief Classe principal que implementa o "domínio de negócio" (
11  *      a lógica de engenharia).
12  *
13  * Esta classe é responsável por implementar a equação de Buckley-
14  *      Leverett.
15  * Ela depende de uma abstração (ICurvasPermeabilidade) e não de
16  *      uma
17  * implementação concreta, seguindo o Princípio da Inversão de
18  *      Dependência.
19  */
20
21 class CalculadoraFluxoFracionario {
22 private:
23     /// Viscosidade do Óleo (cPoise)
24     double _viscosidadeOleo;
25
26     /// Viscosidade da Água (cPoise)
27     double _viscosidadeAgua;
28
29     /// Ponteiro para o modelo de Kr (Strategy Pattern)
30     ICurvasPermeabilidade* _modeloKr;
31
32 public:

```

```

28  /**
29   * @brief Construtor da Calculadora.
30   * Recebe as viscosidades e o modelo de Kr via Injeção de
      Dependência.
31   * @param mu_o Viscosidade do Óleo (cPoise).
32   * @param mu_w Viscosidade da Água (cPoise).
33   * @param modelo Um ponteiro para um objeto que implementa
      ICurvasPermeabilidade.
34   */
35  CalculadoraFluxoFracionario(double mu_o, double mu_w,
      ICurvasPermeabilidade* modelo);
36
37  /**
38   * @brief Calcula um único ponto da curva de fluxo fracionário
      .
39   * @param sw A saturação de água para a qual o Fw será
      calculado.
40   * @return O valor do fluxo fracionário (fw).
41   */
42  double calcularFw(double sw) const;
43
44  /**
45   * @brief Gera a curva completa de Fw vs Sw, iterando sobre a
      saturação.
46   * @param passo O incremento de Saturação (ex: 0.01 para 1%).
47   * @return Um mapa (std::map) contendo os pares (Sw, Fw) da
      curva.
48   */
49  std::map<double, double> gerarCurvaCompleta(double passo)
      const;
50 };
51
52 #endif

```

Apresenta-se na listagem 6.7 a implementação da classe. Note que ela opera apenas contra a interface ICurvasPermeabilidade, garantindo baixo acoplamento.

Listagem 6.7: CalculadoraFluxoFracionario.cpp.

```

1  #include "CalculadoraFluxoFracionario.h"
2  #include <cmath>      // Para std::pow
3  #include <stdexcept>  // Para std::runtime_error
4  #include <limits>     // Para checagem de divisão por zero
5
6  /**
7   * @brief Construtor da Calculadora.
8   * @param mu_o Viscosidade do Óleo.
9   * @param mu_w Viscosidade da Água.
10  * @param modelo Ponteiro para o modelo de Kr (injetado).
11  */
12  CalculadoraFluxoFracionario::CalculadoraFluxoFracionario(double
      mu_o, double mu_w, ICurvasPermeabilidade* modelo)
13  : _viscosidadeOleo(mu_o), _viscosidadeAgua(mu_w), _modeloKr(modelo)
      {
14
15      // Validação de segurança

```

```

16     if (modelo == nullptr) {
17         throw std::runtime_error("Erro: Calculadora recebeu um
            modelo de permeabilidade nulo.");
18     }
19     if (mu_o <= 0 || mu_w <= 0) {
20         throw std::runtime_error("Erro: Viscosidades devem ser
            positivas.");
21     }
22 }
23
24 /**
25  * @brief Calcula o valor do fluxo fracionário (fw).
26  * @param sw Saturação de água.
27  * @return O valor de fw.
28  */
29 double CalculadoraFluxoFracionario::calcularFw(double sw) const {
30
31     // 1. Pedir os valores de Kr para o modelo (Strategy Pattern)
32     double krw = _modeloKr->getKrw(sw);
33     double kro = _modeloKr->getKro(sw);
34
35     // 2. Implementar a Equação de Buckley-Leverett
36
37     // Razão de mobilidade da água: Lambda_w = krw / mu_w
38     double lambda_w = krw / _viscosidadeAgua;
39
40     // Razão de mobilidade do óleo: Lambda_o = kro / mu_o
41     double lambda_o = kro / _viscosidadeOleo;
42
43     // Mobilidade Total: Lambda_t = Lambda_w + Lambda_o
44     double lambda_t = lambda_w + lambda_o;
45
46     // Fórmula do Fluxo Fracionário: fw = Lambda_w / Lambda_t
47
48     // Checagem de segurança para divisão por zero
49     if (lambda_t < std::numeric_limits<double>::epsilon()) {
50         // Se a mobilidade total é zero (ambos krw e kro são 0),
51         // o fluxo fracionário também é zero.
52         return 0.0;
53     }
54
55     return lambda_w / lambda_t;
56 }
57
58 /**
59  * @brief Gera a curva completa de fw vs Sw.
60  * @param passo O incremento de Saturação (ex: 0.01 para 1%).
61  * @return Um mapa (map) contendo os pares (Sw, Fw).
62  */
63 std::map<double, double> CalculadoraFluxoFracionario::
    gerarCurvaCompleta(double passo) const {
64     std::map<double, double> curva;
65
66     for (double sw = 0.0; sw <= 1.0; sw += passo) {
67         curva[sw] = calcularFw(sw);
68     }
69     // Garante que o ponto final (1.0) seja sempre calculado

```

```

70     curva[1.0] = calcularFw(1.0);
71
72     return curva;
73 }

```

- Classe Gnuplot

Apresenta-se na listagem 6.8 o arquivo de cabeçalho da classe utilitária Gnuplot, que atua como uma Facade para a plotagem.

Listagem 6.8: Gnuplot.h.

```

1  #ifndef GNUPLOT_H
2  #define GNUPLOT_H
3
4  #include <map>
5  #include <string>
6
7  /**
8   * @class Gnuplot
9   * @brief Classe utilitária que implementa o Padrão de Projeto
10    Facade.
11
12   * Esta classe esconde toda a complexidade de lidar com o Gnuplot
13   * (criar arquivos temporários, formatar scripts, chamar o system
14   * ())
15   * por trás de um único método estático.
16   */
17  class Gnuplot {
18  public:
19      /**
20       * @brief Plota um mapa de dados (x, y) usando o Gnuplot.
21       * Este é um método estático, pois não precisa de um estado de
22       * instância.
23       * @param dados Um std::map<double, double> onde a chave é o
24       * eixo X (Sw) e o valor é o eixo Y (Fw).
25       * @param titulo O título que aparecerá no topo do gráfico.
26       */
27      static void plotarCurva(const std::map<double, double>& dados,
28                             const std::string& titulo);
29 };
30
31 #endif

```

Apresenta-se na listagem 6.9 a implementação da classe, incluindo a chamada `system()` para o processo externo do Gnuplot.

Listagem 6.9: Gnuplot.cpp.

```

1  #include "Gnuplot.h"
2  #include <iostream>
3  #include <fstream>
4  #include <cstdlib> // Para system()
5
6  // TODO: Documentar com JAVADOC/Doxygen
7  void Gnuplot::plotarCurva(const std::map<double, double>& dados,
8                             const std::string& titulo) {
9      std::cout << "DEBUG: Chamando Gnuplot...\n";

```

```

9
10 // Nomes dos arquivos temporários
11 std::string tempDados = "temp_data.csv";
12 std::string tempScript = "temp_script.gp";
13
14 // — 1. Salvar dados no arquivo .csv —
15 std::ofstream arqDados(tempDados);
16 if (!arqDados.is_open()) {
17     std::cerr << "Erro: Não foi possível criar arquivo de
18         dados temporário.\n";
19     return;
20 }
21 arqDados << "#Sw, Fw\n";
22 for (const auto& par : dados) {
23     arqDados << par.first << ", " << par.second << "\n";
24 }
25 arqDados.close();
26
27 // — 2. Criar script do Gnuplot —
28 std::ofstream arqScript(tempScript);
29 if (!arqScript.is_open()) {
30     std::cerr << "Erro: Não foi possível criar script Gnuplot
31         temporário.\n";
32     return;
33 }
34 arqScript << "set title " << titulo << "\n";
35 arqScript << "set xlabel 'Saturacao de Agua (Sw)' \n";
36 arqScript << "set ylabel 'Fluxo Fracionario de Agua (Fw)' \n";
37 arqScript << "set grid\n";
38 arqScript << "set key top left\n";
39 arqScript << "set datafile separator ',' \n";
40 arqScript << "plot " << tempDados << " with lines title '
41     Curva Fw'\n";
42 arqScript << "pause 1 'Pressione Enter para fechar'\n"; //
43     Mantém a janela aberta
44 arqScript.close();
45
46 // — 3. Executar Gnuplot —
47 // Este comando supõe que 'gnuplot' está no PATH do sistema
48 std::string comando = "gnuplot " + tempScript;
49 std::system(comando.c_str());
50
51 // — 4. (Opcional) Limpar arquivos temporários —
52 // std::remove(tempDados.c_str());
53 // std::remove(tempScript.c_str());
54 }

```

- Classe Simulador

Apresenta-se na listagem 6.10 o arquivo de cabeçalho da classe Simulador, que atua como o orquestrador da aplicação.

Listagem 6.10: Simulador.h.

```

1 #ifndef SIMULADOR_H
2 #define SIMULADOR_H
3
4 #include <string>

```

```

5
6 /**
7  * @class Simulador
8  * @brief O orquestrador da aplicação.
9  *
10 * Esta classe é responsável por controlar o fluxo de execução do
    programa:
11 * ler o arquivo de entrada, instanciar os objetos corretos
12 * (fazendo o 'new' dos modelos) e coordenar as chamadas
13 * para a calculadora e o plotter.
14 */
15 class Simulador {
16 public:
17     /**
18      * @brief Ponto de entrada principal da lógica do simulador.
19      * @param arquivoEntrada O caminho (path) para o arquivo de
        configuração .txt.
20      */
21     void executar(const std::string& arquivoEntrada);
22 };
23
24 #endif

```

Apresenta-se na listagem 6.11 a implementação da classe, responsável por ler o arquivo de entrada, instanciar os objetos corretos e injetar as dependências.

Listagem 6.11: Simulador.cpp

```

1 #include "Simulador.h"
2 #include "CalculadoraFluxoFracionario.h"
3 #include "CurvasPermeabilidadeTabelada.h"
4 #include "CurvasPermeabilidadeCorey.h"
5 #include "Gnuplot.h"
6
7 #include <iostream>
8 #include <fstream> // Para ler arquivos (ifstream)
9 #include <sstream> // Para processar strings (stringstream)
10 #include <stdexcept> // Para lançar erros (runtime_error)
11 #include <map>
12
13 /**
14  * @brief Executa a simulação completa.
15  * * Este método orquestra todo o processo:
16  * 1. Lê o arquivo de entrada para definir os parâmetros.
17  * 2. Instancia o modelo de permeabilidade correto (Tabelado ou
    Corey).
18  * 3. Delega o carregamento de dados detalhados para o modelo.
19  * 4. Instancia a calculadora.
20  * 5. Gera a curva de fluxo fracionário.
21  * 6. Chama o Gnuplot para exibir o resultado.
22  * * @param arquivoEntrada O caminho para o arquivo de
        configuração .txt.
23  */
24 void Simulador::executar(const std::string& arquivoEntrada) {
25     std::cout << "Iniciando simulador...\n";
26

```



```

27 // — 1. Leitura e Parsing do Arquivo de Entrada —
28
29 double mu_o = -1.0;
30 double mu_w = -1.0;
31 std::string tipoModelo;
32 ICurvasPermeabilidade* modelo = nullptr;
33
34 std::ifstream arq(arquivoEntrada);
35 if (!arq.is_open()) {
36     throw std::runtime_error("Erro fatal: Não foi possível
37         abrir o arquivo: " + arquivoEntrada);
38 }
39
40 std::cout << "Lendo arquivo de configuracao: " <<
41     arquivoEntrada << "\n";
42 std::string linha;
43 std::string palavraChave;
44
45 while (std::getline(arq, linha)) {
46     // Ignora linhas vazias ou comentários
47     if (linha.empty() || linha[0] == '#') {
48         continue;
49     }
50
51     std::stringstream ss(linha);
52     ss >> palavraChave;
53
54     if (palavraChave == "VISC_OLEO") {
55         ss >> mu_o;
56     } else if (palavraChave == "VISC_AGUA") {
57         ss >> mu_w;
58     } else if (palavraChave == "MODELO_KR") {
59         ss >> tipoModelo;
60     }
61 }
62
63 // — 2. Validação e Instanciação do Modelo —
64
65 if (mu_o <= 0 || mu_w <= 0) {
66     throw std::runtime_error("Erro: Viscosidades do óleo ou da
67         água não definidas no arquivo.");
68 }
69
70 if (tipoModelo == "TABELADO") {
71     std::cout << "Modelo selecionado: TABELADO\n";
72     modelo = new CurvasPermeabilidadeTabelada();
73 } else if (tipoModelo == "COREY") {
74     std::cout << "Modelo selecionado: COREY\n";
75     modelo = new CurvasPermeabilidadeCorey();
76 } else {
77     throw std::runtime_error("Erro: MODELO_KR não reconhecido.
78         Use TABELADO ou COREY.");
79 }
80
81 // — 3. Delegar Carregamento de Dados —
82 // O modelo agora lê o *mesmo* arquivo para pegar seus dados
83 // específicos

```

```

79     modelo->carregarDados(arquivoEntrada);
80
81     // — 4. Criar Calculadora (Injeção de Dependência) —
82     CalculadoraFluxoFracionario calc(mu_o, mu_w, modelo);
83
84     // — 5. Gerar Curva —
85     std::cout << "Calculando curva...\n";
86     std::map<double, double> curva = calc.gerarCurvaCompleta(0.01)
87         ; // passo de 1%
88
89     // — 6. Plotar —
90     std::cout << "Plotando resultados...\n";
91     Gnuplot::plotarCurva(curva, "Curva de Fluxo Fracionario (
92         Buckley–Leverett)");
93
94     // — 7. Limpeza da Memória —
95     delete modelo;
96     modelo = nullptr;
97
98     std::cout << "Simulacao concluida.\n";
99 }

```

- Programa Principal `main.cpp`

Apresenta-se na listagem 6.12 o arquivo `main.cpp`, que contém o ponto de entrada do programa. Sua única responsabilidade é instanciar e executar o Simulador.

Listagem 6.12: `main.cpp`.

```

1  #include "Simulador.h"
2  #include <iostream>
3
4  int main(int argc, char* argv[]) {
5      // Verifica se o usuário passou o nome do arquivo de entrada
6      if (argc < 2) {
7          std::cerr << "Erro: Por favor, forneça o nome do arquivo de
8              entrada.\n";
9          std::cerr << "Uso: ./fw_calc <
10             caminho_para_arquivo_de_entrada>\n";
11             return 1; // Retorna um código de erro
12         }
13
14         std::string arquivoEntrada = argv[1];
15
16         try {
17             Simulador sim;
18             sim.executar(arquivoEntrada);
19         } catch (const std::exception& e) {
20             std::cerr << "Uma exceção ocorreu: " << e.what() << '\n';
21             return 1;
22         }
23
24         return 0; // Sucesso
25     }

```

Capítulo 7

Teste

Todo projeto de engenharia passa por uma etapa de testes. Neste capítulo apresentamos alguns testes do software desenvolvido. Estes testes devem dar resposta aos diagramas de caso de uso inicialmente apresentados (diagramas de caso de uso geral e específicos).

7.1 Teste 1: Validando o Ciclo 1 - Modelo Tabe-lado

Descrição do Teste:

- O que está sendo testado? Este teste valida o funcionamento básico do núcleo numérico e o algoritmo de interpolação linear. O objetivo é garantir que o software consiga ler uma tabela de permeabilidade relativa genérica e calcular valores intermediários corretamente.
- Como será realizado? O software é executado com o arquivo de entrada `test/Teste-01.in`, que contém uma tabela simplificada com 7 pontos de k_{rw} e k_{ro} .
- Como será validado? A validação é feita pela análise da curva gerada no arquivo `output.csv`. Verifica-se se a curva é contínua e se os pontos calculados correspondem a uma interpolação linear suave entre os pontos de entrada, sem saltos ou descontinuidades numéricas.

Arquivo de entrada de dados

Apresenta-se a seguir o arquivo de entrada `Teste-01.in`, Listagem 7.1, que utiliza o modelo tabelado.

Listagem 7.1: Teste-01.in.

```
# Exemplo de arquivo de entrada para modelo tabelado
```

```

VISC_OLEO 1.5
VISC_AGUA 0.8
MODELO_KR TABELADO #SW/KRW/KRO
DADOS_KR_INICIO
0.20 0.00 0.90
0.30 0.05 0.75
0.40 0.12 0.50
0.50 0.20 0.30
0.60 0.30 0.15
0.70 0.40 0.05
0.80 0.50 0.00
FIM_DADOS

```

Arquivo de saída de dados (resultados da tela)

O software é de linha de comando e não gera saída na tela, exceto por mensagens de status ou erro. O resultado principal é o arquivo `output.csv`, que conterá duas colunas: S_w e f_w . *Reservoir Engineering Handbook* de Tarek Ahmed [Ahmed, 2010],

Figuras

A Figura 7.1 mostra a execução do programa no terminal. A Figura 7.2 mostra o gráfico gerado a partir do `output.csv`, comprovando a geração da curva "S" característica.

A Figura 7.2 apresenta o gráfico gerado. A curva suavizada confirma que a interpolação linear entre os pontos fornecidos está funcionando corretamente, sem descontinuidades.

```

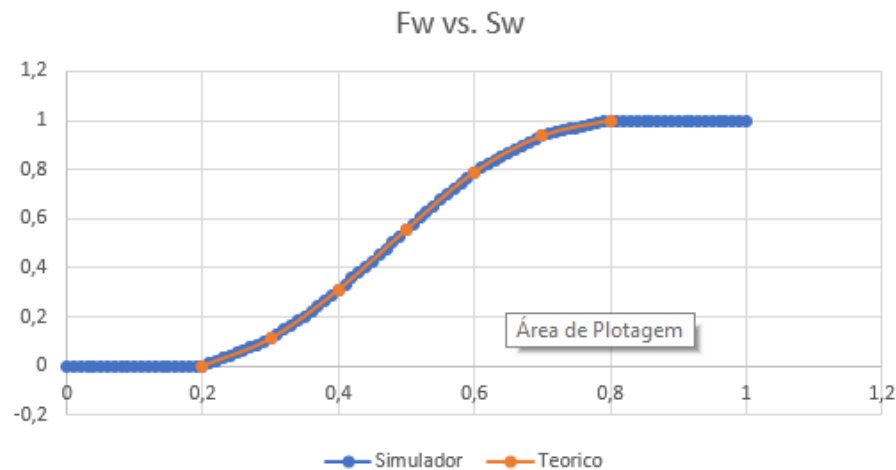
C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ cd "/C:/Users/Fabia/OneDrive/Documentos/Disciplinas UENF 2025.2/Disciplina de Projeto/projeto-git/LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1/"
$ cd "/C:/Users/Fabia/OneDrive/Documentos/Disciplinas UENF 2025.2/Disciplina de Projeto/projeto-git/LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1/"
$ make clean
Arquivos de compilação limpos.

C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ make
g++ -std=c++17 -Wall -g -Isrc -c src/main.cpp -o obj/main.o
g++ -std=c++17 -Wall -g -Isrc -c src/Simulador.cpp -o obj/Simulador.o
g++ -std=c++17 -Wall -g -Isrc -c src/CalculadoraFluxoFracionario.cpp -o obj/CalculadoraFluxoFracionario.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeTabelada.cpp -o obj/CurvasPermeabilidadeTabelada.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeCorey.cpp -o obj/CurvasPermeabilidadeCorey.o
g++ -std=c++17 -Wall -g -Isrc -c src/Gnuplot.cpp -o obj/Gnuplot.o
g++ -std=c++17 -Wall -g -Isrc -o bin/fw_calc obj/main.o obj/Simulador.o obj/CalculadoraFluxoFracionario.o obj/CurvasPermeabilidadeTabelada.o obj/CurvasPermeabilidadeCorey.o obj/Gnuplot.o
Compilação concluída! Executável está em bin/fw_calc

C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ ./bin/fw_calc test/Teste-01.in
Iniciando simulador...
Lendo arquivo de configuracao: test/Teste-01.in
Modelo selecionado: TABELADO
DEBUG: 7 pontos de Kr tabelados foram carregados.
Calculando curvas...
Plotando resultados...
DEBUG: Chamando Gnuplot...
Pressione Enter para fechar

```

Figura 7.1: Tela do programa mostrando terminal executando.

Figura 7.2: Tela do programa mostrando gráfico (*Excel*) gerado a partir do output.csv.

7.2 Teste 2: Validando o Ciclo 2 - Modelo Corey

Descrição do Teste:

- O que está sendo testado? Este teste valida a arquitetura de software (Padrão *Strategy*) e a implementação matemática do modelo analítico de Corey. O objetivo é provar que o sistema consegue trocar de modelo de cálculo dinamicamente apenas alterando o arquivo de entrada.
- Como será realizado? O software é executado com o arquivo `test/Teste-02.in`, que utiliza a palavra-chave `MODELO_KR COREY` e define os expoentes $n_w = 2.0$ e $n_o = 2.5$.
- Como será validado? A validação é visual e analítica. A curva resultante deve apresentar o perfil exponencial característico das equações de Corey, diferindo das linhas retas do modelo tabelado. Além disso, os pontos finais (saturação irreduzível de água e óleo residual) devem coincidir exatamente com os parâmetros S_{wir} e S_{orw} fornecidos no arquivo.

Arquivo de entrada de dados

Apresenta-se a seguir o arquivo de entrada `Teste-02.in`, Listagem 7.2.

Listagem 7.2: Teste-02.in.

```
# Exemplo de arquivo de entrada para modelo Corey
VISC_OLEO 2.0
VISC_AGUA 1.0
MODELO_KR COREY
COREY_SWIR      0.15
COREY_SORW      0.20
COREY_KRW_MAX   0.5
COREY_KRO_MAX   0.9
COREY_NW        2.0
COREY_NO        2.5
```

Arquivo de saída de dados (resultados da tela)

Similar ao Teste 1, o resultado principal é o arquivo `output.csv`.

Figuras

A Figura 7.3 mostra o resultado gráfico. A curva apresenta o comportamento exponencial característico do modelo de Corey, confirmando a correta implementação das equações de potência.

A curva gerada apresenta a concavidade característica das funções de potência, confirmando que os expoentes $n_w = 2.0$ e $n_o = 2.5$ foram aplicados corretamente na fórmula analítica.

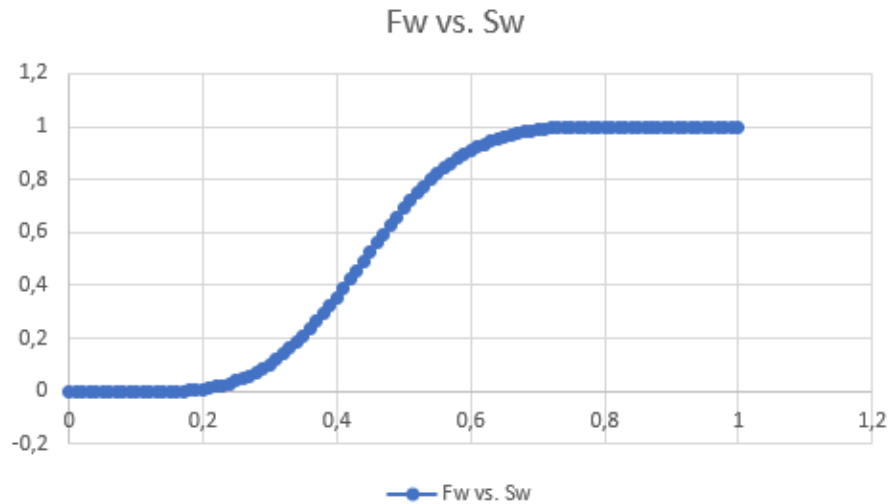


Figura 7.3: Tela do programa mostrando curva do modelo de Corey.

7.3 Teste 3: Validação com Literatura (Tarek Ahmed)

Descrição do Teste

- O que está sendo testado? Este é o teste de validação científica do projeto. O objetivo é comprovar a precisão física dos resultados comparando-os com um problema clássico da literatura de engenharia de reservatórios.
- Como será realizado? O software é executado com o arquivo `test/Teste-Tarek-1cp.in`. Este arquivo reproduz os parâmetros exatos do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [Ahmed, 2010], considerando a viscosidade do óleo de 1.0 cp.
- Como será validado?
 1. Numérica: Os valores do arquivo de saída `output.csv` são comparados com a coluna " $\mu_o = 1.0$ " da tabela de resultados na página 950 do livro 1. O erro absoluto deve ser desprezível.
 2. Gráfica: A curva gerada (Figura 7.4) é comparada visualmente com a Figura 7.5 do livro, buscando a reprodução fiel da curva "S".

Arquivo de entrada de dados

Apresenta-se a seguir o conteúdo do arquivo `Teste-Tarek-1cp.in` utilizado para esta simulação, Listagem 7.3.

Listagem 7.3: Teste-Tarek-1cp.in.

```
# Teste de Validação – Exemplo 14–5 do Tarek Ahmed
# Viscosidade do Oleo = 1.0 cp
# Viscosidade da Agua = 0.5 cp

VISC_OLEO 1.0
VISC_AGUA 0.5

MODELO_KR TABELADO

# Dados lidos da Figura 14–15 (Tabela pg. 950)
DADOS_KR_INICIO
0.24 0.00 0.95
0.30 0.01 0.89
0.40 0.04 0.74
0.50 0.09 0.45
0.60 0.17 0.19
0.65 0.28 0.12
0.70 0.22 0.06
0.75 0.36 0.03
0.78 0.41 0.00
FIM_DADOS
```

Figuras

A Figura 7.4 apresenta o gráfico gerado automaticamente pelo software após a execução do teste.

A análise comparativa com a Figura 7.5 demonstra que a curva obtida reproduz com exatidão o comportamento esperado descrito por Ahmed [Ahmed, 2010]. Os pontos de saturação inicial de água ($S_{wi} \approx 0.24$) e saturação de óleo residual ($S_{or} \approx 0.22$, ou seja, $S_w \approx 0.78$) coincidem perfeitamente com os limites da tabela de entrada, e a curvatura "S" reflete corretamente a variação da razão de mobilidade ao longo do domínio de saturação. Isso confirma que o núcleo numérico da ferramenta é robusto e confiável para aplicações de engenharia.

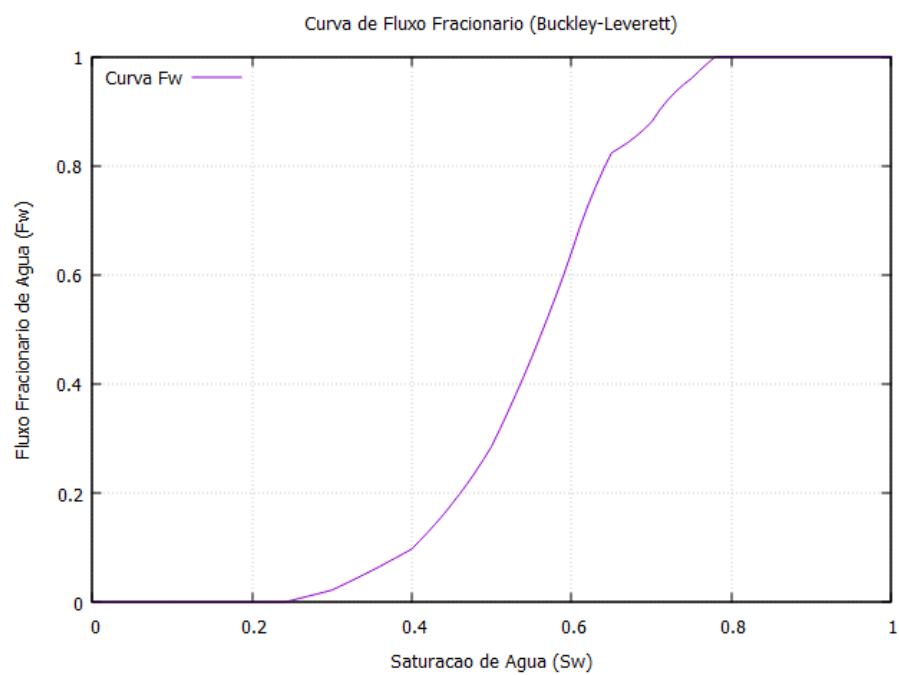


Figura 7.4: Tela do programa mostrando curva do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [Ahmed, 2010] (com $\mu_o = 1.0$).

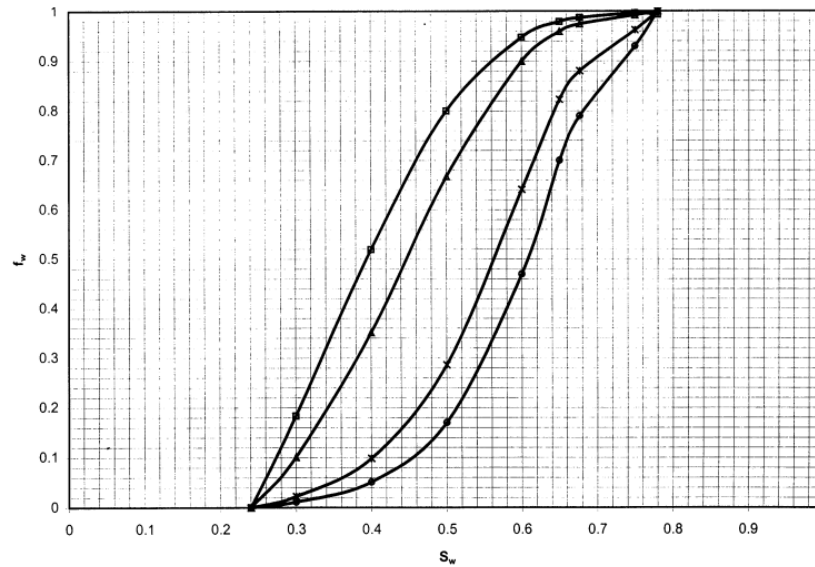


Figure 14-16. Effect of μ_o on f_w .

Figura 7.5: Curva do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [Ahmed, 2010] (com $\mu_o = 0.5, \mu_o = 1.0, \mu_o = 5$ e $\mu_o = 10$).

Observar que, a Figura 7.4 representa a curva com viscosidade do óleo igual a 1.0 cp ($\mu_o = 1.0$), para comparação com a devida curva da imagem 7.5.

7.4 Teste 4: Validação da Camada de Apresentação (Gnuplot)

Descrição do Teste:

- O que está sendo testado? Este teste valida o requisito não funcional de usabilidade e a integração com a ferramenta externa Gnuplot (Padrão Facade).
- Como será realizado? O teste consiste na execução bem-sucedida de qualquer um dos cenários anteriores em um ambiente *Desktop* (*Windows* ou *Linux*).
- Como será validado? O teste é considerado aprovado se, após o cálculo, uma janela gráfica do Gnuplot for aberta automaticamente, exibindo a curva correta com títulos, eixos nomeados e grade (*grid*) ativada, sem a

necessidade de intervenção manual do usuário para plotar os dados. A persistência da janela (ela não deve fechar sozinha imediatamente) também é um critério de sucesso.

Arquivo de entrada de dados

Este teste reutiliza o arquivo `Teste-01.in`. Além da saída padrão (`output.csv`), o resultado esperado é uma ação do sistema: a abertura de uma janela de gráfico.

Figuras

A Figura 7.6 mostra uma captura de tela a janela do Gnuplot que foi aberta automaticamente pelo software, exibindo o resultado gráfico.

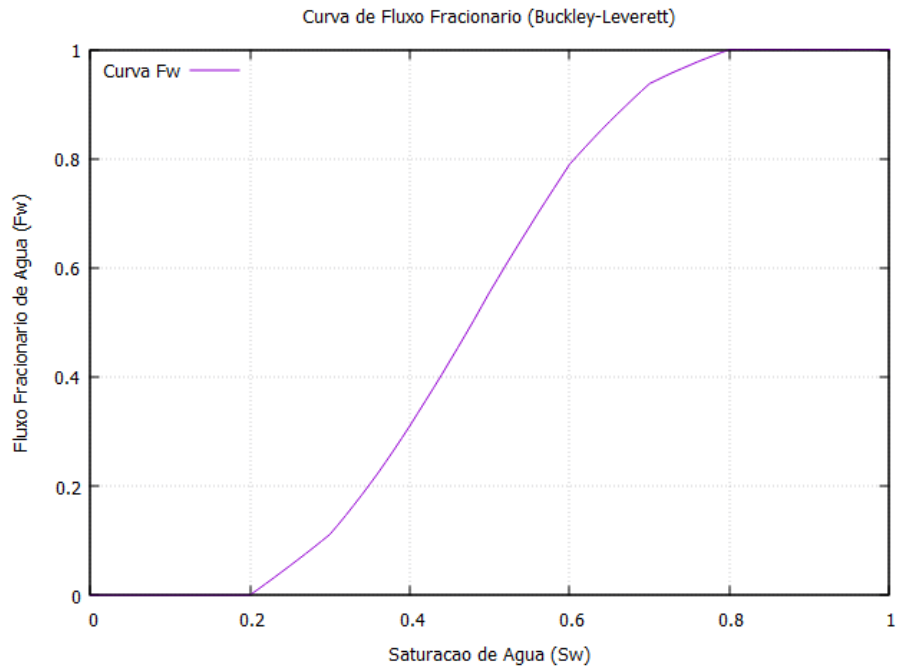


Figura 7.6: Tela do programa mostrando captura de tela a janela do Gnuplot.

Capítulo 8

Documentação para o Desenvolvedor

8.1 Dependências para compilar o software

Para compilar e executar o software "Ferramenta de Fluxo Fracionário", é necessário configurar um ambiente de desenvolvimento C++ compatível. O projeto foi desenvolvido e testado no ambiente MSYS2 (*MinGW* 64-bit) no *Windows*.

As seguintes dependências são obrigatórias:

1. Compilador C++ (g++): Deve suportar o padrão C++17.
 2. GNU *Make*: Ferramenta de automação de compilação utilizada pelo `Makefile` do projeto.
 3. Gnuplot: Ferramenta de plotagem externa, necessária para a visualização dos gráficos (requisito de execução).
 4. Doxygen (Opcional): Para geração da documentação automática.
- Instalação das Dependências (via MSYS2):

No terminal `MINGW64`, execute os seguintes comandos, Listagem 8.1, para instalar todas as ferramentas necessárias:

Listagem 8.1: Comando para instalar Compilador, Make e Gnuplot no MSYS2..

```
pacman -S mingw-w64-x86_64-toolchain mingw-w64-x86_64-gnuplot
```

8.2 Como gerar a documentação usando doxygen

A documentação do código-fonte foi escrita utilizando o padrão Javadoc. Isso permite que a ferramenta Doxygen analise os arquivos `.h` e gere um site HTML navegável com a descrição de todas as classes e métodos.

- Passo a Passo para Gerar a Documentação:
- Instale o Doxygen: Caso não tenha instalado, use o comando: `gen}`
- Navegue até a pasta do código: Abra o terminal na raiz do projeto e entre na pasta `src/`
- Execute o Doxygen, Listagem 8.2: O arquivo de configuração Doxyfile já está configurado no projeto. Basta rodar o comando:

Listagem 8.2: Instalação do Doxygen.

```
pacman -S mingw-w64-x86_64-doxygen
```

- Visualize o Resultado, Listagem 8.3: A documentação será gerada na pasta `html/`. Para visualizar, abra o arquivo `index.html` no seu navegador:

Listagem 8.3: Gerando a documentação HTML.

```
doxygen
```

A Figura 8.1 ilustra a página inicial da documentação gerada, onde é possível navegar pela hierarquia de classes - Figura 8.2 - e ver os diagramas de colaboração. Na figura 8.3 a página para `CalculadoraFluxoFracionario Class Reference` e na Figura 8.4 o arquivo `CalculadoraFluxoFracionario.h`.

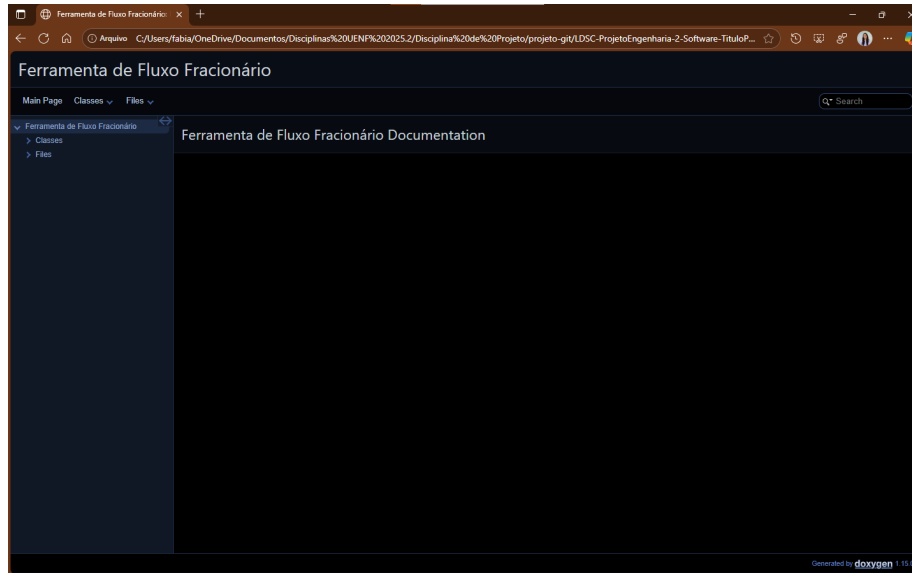


Figura 8.1: Página inicial.

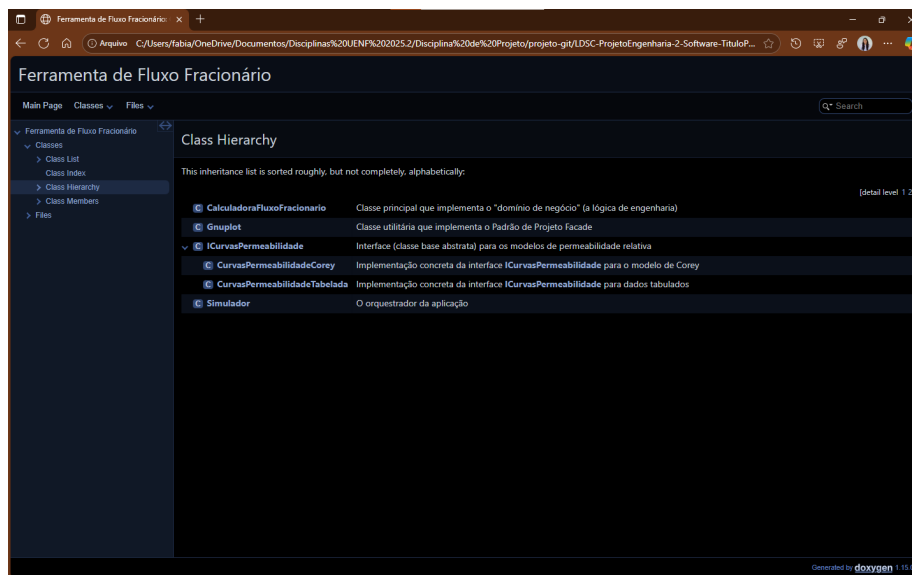
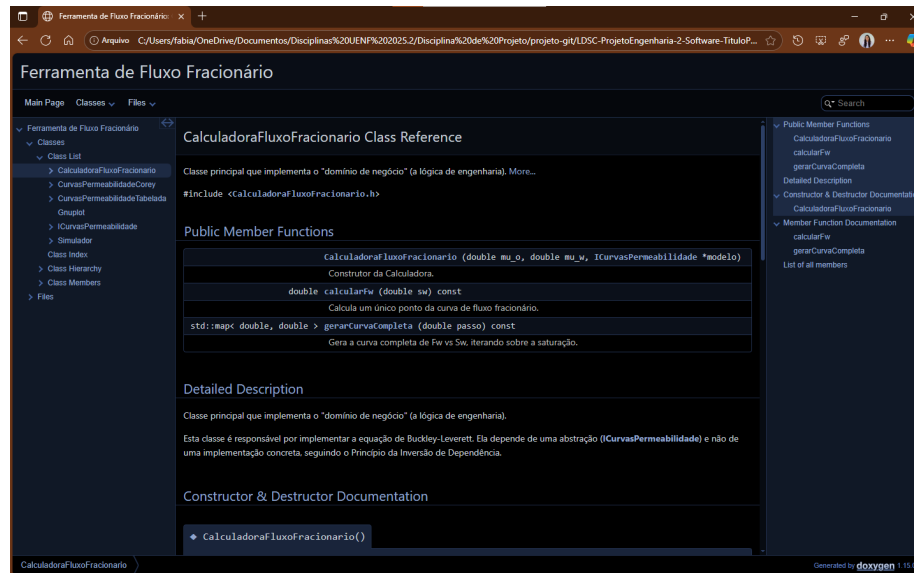
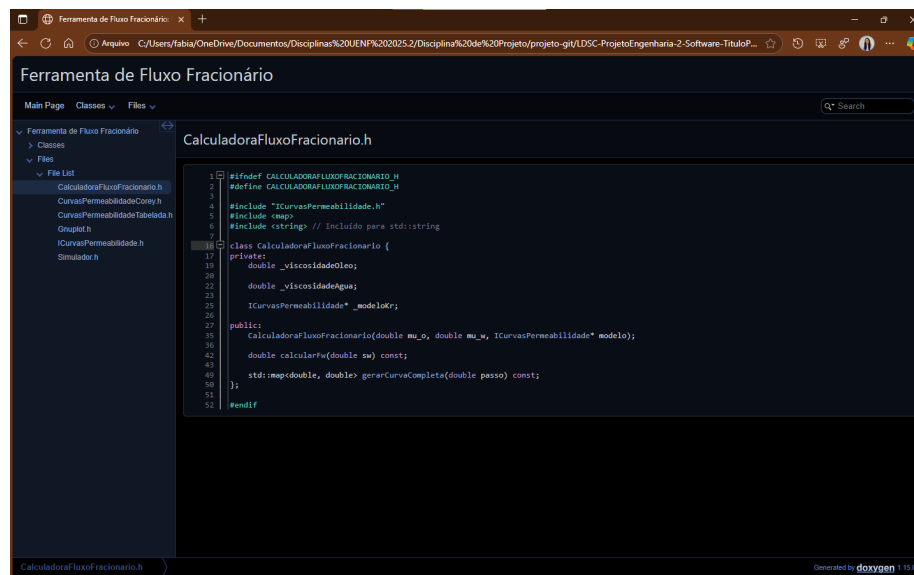


Figura 8.2: Hierarquia de classes.

Figura 8.3: Página para `CalculadoraFluxoFracionario` *Class Reference*.Figura 8.4: Arquivo `CalculadoraFluxoFracionario.h`.

Referências Bibliográficas

- [Ahmed, 2010] Ahmed, T. (2010). *Reservoir Engineering Handbook*. Gulf Publishing Company, 4th edition.
- [Buckley and Leverett, 1942] Buckley, S. E. and Leverett, M. C. (1942). Mechanism of Fluid Displacement in Sands. *Transactions of the AIME*, 146:107–116.
- [Corey, 1954] Corey, A. T. (1954). The Interrelation Between Gas and Oil Relative Permeabilities. *Producers Monthly*, 19(1):38–41.
- [Lake et al., 2010] Lake, L. W. et al. (2010). *Fundamentals of Enhanced Oil Recovery*. Society of Petroleum Engineers.

Apêndice A

Formato do Arquivo de Entrada

O software utiliza um arquivo de entrada de texto `.txt` baseado em palavras-chave. O parser identifica a palavra-chave (ex: `VISC_OLEO`) e lê o valor na mesma linha.

A palavra-chave `MODELO_KR` define qual modelo de permeabilidade será usado: `TABELADO` ou `COREY` .

A.1 Exemplo de Arquivo de Entrada (Modelo Tabelado)

A seguir a Listagem A.1, com um exemplo de arquivo de entrada para modelo tabelado.

Listagem A.1: Exemplo de arquivo de entrada para modelo tabelado.

```
# Exemplo de arquivo de entrada para modelo tabelado
# Comentários são ignorados

VISC_OLEO 1.5
VISC_AGUA 0.8

MODELO_KR TABELADO

# Definição da tabela Sw, Krw, Kro
# Os dados são lidos até a palavra-chave FIM_DADOS
DADOS_KR_INICIO
0.20 0.00 0.90
0.30 0.05 0.75
0.40 0.12 0.50
```

```
0.50 0.20 0.30
0.60 0.30 0.15
0.70 0.40 0.05
0.80 0.50 0.00
FIM_DADOS
```

A.2 Exemplo de Arquivo de Entrada (Modelo Corey)

A seguir a Listagem A.2, com um exemplo de arquivo de entrada para modelo Corey.

Listagem A.2: Exemplo de arquivo de entrada para modelo Corey.

```
# Exemplo de arquivo de entrada para modelo Corey
# Comentários são ignorados

VISC_OLEO 2.0
VISC_AGUA 1.0

MODELO_KR COREY

# Parâmetros da correlação de Corey
COREY_SWIR      0.15
COREY_SORW      0.20
COREY_KRW_MAX   0.5
COREY_KRO_MAX   0.9
COREY_NW         2.0
COREY_NO         2.5
```

Apêndice B

Guia de Compilação e Execução

Este apêndice descreve os passos exatos para compilar e executar o software "Ferramenta de Fluxo Fracionário" em um ambiente *Windows*, utilizando o terminal MSYS2 MINGW64.

B.1 Pré-requisitos

Certifique-se de que o ambiente MSYS2 está instalado com os pacotes `mingw-w64-x86_64-toolchain` (que inclui `g++` e `make`) e `mingw-w64-x86_64-gnuplot`.

B.2 Compilação do Projeto

1. Abra o terminal MSYS2 MINGW64.
2. Navegue até a pasta raiz do projeto (onde está o arquivo `Makefile`).
Exemplo, Listagem B.1:

Listagem B.1: Comando para navegar até a pasta do projeto.

```
cd "/c/Users/SeuUsuario/Documents/  
ProjetoFluxoFracionario/"
```

3. Execute o comando de limpeza para garantir uma compilação limpa, Listagem B.2:

Listagem B.2: Limpando arquivos antigos.

```
make clean
```

4. Compile o projeto usando o comando `make`. O `Makefile` irá gerenciar a compilação de todos os arquivos `.cpp` e gerar o executável na pasta `bin/`, Listagem B.3.

Listagem B.3: Comando de compilação.

```
make
```

B.3 Execução e Teste

Após a compilação bem-sucedida (mensagem "Compilação concluída!"), o programa pode ser executado.

1. Para rodar o Teste 1 (Modelo Tabelado), execute, Listagem B.4:

Listagem B.4: Executando o Teste 1.

```
./bin/fw_calc test/Teste-01.in
```

2. Para rodar o Teste de Validação (Dados de Tarek Ahmed,[Ahmed, 2010]), execute, Listagem B.5:

Listagem B.5: Executando a Validação com Literatura.

```
./bin/fw_calc test/Teste-Tarek-1cp.in
```

Se o ambiente estiver configurado corretamente, uma janela do Gnuplot abrirá automaticamente exibindo o gráfico da curva de fluxo fracionário.

Índice Remissivo

A

Análise orientada a objeto, 17
AOO, 17

C

Casos de uso, 10
Ciclo construção, 35
Ciclos de Planejamento/Detalhamento,
32
colaboração, 21
comunicação, 21
Concepção, 7

D

Diagrama de colaboração, 21
Diagrama de componentes, 27
Diagrama de estado, 22
Diagrama de execução, 28
Diagrama de implantação, 28
Diagrama de sequência, 19

E

Elaboração, 12
Especificação, 8
especificação, 8
estado, 22
Eventos, 19

I

Implementação, 35

M

Mensagens, 19

P

POO, 26
Projeto orientado a objeto, 26

R

Requisitos, 9
Requisitos funcionais, 9
Requisitos não funcionais, 9